



भारतीय संगणक एवं प्रौद्योगिकी संस्थान

INDIAN INSTITUTE OF COMPUTING AND TECHNOLOGY

AFFILIATED: I-STEM, OFFICE OF THE PRINCIPAL SCIENTIFIC ADVISER TO THE GOVERNMENT OF INDIA

PhishGuard AI: An AI-Driven Phishing Email Detection System Using Hybrid NLP and Structural Metadata Analysis



Student Name: A.P.Anirudh

Institute Name: Vellore Institute of Technology, Chennai

Institute Roll no.: 24BEC1158

Enrollment no.: 595638

PhishGuard AI: An AI-Driven Phishing Email Detection System Using Hybrid NLP and Structural Metadata Analysis

A.P.Anirudh

IICT Summer Internship 2026 — Project 2

aanirudh549@gmail.com

Abstract—Phishing email remains one of the most persistent initial-access vectors in cybersecurity, and purely lexical detectors that rely on vocabulary alone are increasingly evaded by attackers who imitate legitimate corporate language. This paper presents PhishGuard AI, a phishing email detection system that fuses two complementary feature channels: a 10,000-dimensional TF-IDF representation (unigrams and bigrams) capturing vocabulary and semantic patterns, and a 12-dimensional hand-crafted structural/behavioral metadata vector capturing email “anatomy” signals such as URL count, IP-literal links, urgency and monetary keyword density, punctuation intensity, and HTML markup presence. The two channels are concatenated via sparse matrix stacking into a unified 10,012-dimensional hybrid representation prior to classification. On a benchmark corpus of 18,650 labeled emails (Kaggle Phishing Email Dataset), four classifiers were trained and compared: Complement Naive Bayes, Logistic Regression, Random Forest, and a Multilayer Perceptron (MLP). The hybrid MLP achieved 98.9% accuracy, 98.9% F1-score, and 99.9% ROC-AUC, outperforming a text-only MLP baseline by 1.4 percentage points, confirming that structural metadata carries signal complementary to vocabulary alone. The system is deployed as an interactive Streamlit web application with real-time metadata visualization.

Keywords—Phishing Detection, Natural Language Processing, Email Security, TF-IDF, Multilayer Perceptron, Metadata Features, Cybersecurity, Machine Learning

I. INTRODUCTION

Email remains the most heavily used communication channel in the enterprise, and it is correspondingly the most exploited. Industry threat reporting consistently places

phishing among the leading initial-access vectors: the 2025 Verizon Data Breach Investigations Report attributes 16% of confirmed breaches to phishing and finds that the human element — phishing, stolen credentials, and social engineering combined — is implicated in the majority of incidents [1]. IBM's 2025 Cost of a Data Breach Report similarly identifies phishing as the most common initial attack vector of the year, with breaches that begin with a phishing email costing organizations several million dollars on average to contain [2]. Beyond breach statistics, phishing volume itself continues to climb, with industry trend reports recording hundreds of thousands of unique phishing campaigns per quarter. The attacks themselves have also become harder to catch by eye: modern phishing kits reproduce corporate branding, invoke real employee names harvested from prior breaches or social media, and increasingly draw on generative language models to eliminate the grammatical tells that once made scam emails easy to spot.

Traditional email defenses — static blacklists, sender-reputation checks, and keyword or regular-expression rules — struggle against this evolution because they encode a fixed, enumerable notion of “suspicious,” while attackers only need to vary wording slightly to slip past a rule list. Machine-learning-based classifiers address this brittleness by learning statistical regularities directly from labeled examples, but the majority of published systems represent an email using only its lexical content, typically via bag-of-words or TF-IDF vectors over cleaned message text. A purely lexical model is vulnerable in the opposite direction: it can be defeated by an attacker who simply borrows vocabulary from legitimate corporate correspondence, and it discards a large amount of easily computed, highly diagnostic information that lives outside the vocabulary layer entirely — how many links a message contains, whether those links resolve to a raw IP

address instead of a named domain, how densely the message is packed with urgency language, and whether the email carries raw HTML markup where a legitimate plain-text notification would not.

This paper presents PhishGuard AI, a phishing email detection system built around the hypothesis that combining lexical and structural evidence produces a stronger and more robust classifier than either channel alone. The system extracts a standard 10,000-feature TF-IDF representation over unigrams and bigrams of cleaned email text and pairs it with a 12-dimensional hand-crafted metadata vector that captures structural and behavioral signals — link characteristics, punctuation and capitalization intensity, urgency and monetary keyword density, message length, digit density, and HTML presence. The two representations are concatenated into a single 10,012-dimensional hybrid feature space and used to train and compare four classifiers, of which a Multilayer Perceptron achieves the best performance at 98.9% accuracy. An ablation study isolating the metadata channel's contribution shows a 1.4 percentage-point accuracy gain over a text-only baseline, confirming that structural signal is complementary to, rather than redundant with, lexical content. The system is deployed as an interactive three-tab Streamlit web application that classifies emails in real time and visualizes the metadata feature values as the user types.

The remainder of this paper is organized as follows. Section II reviews related work in phishing and email classification. Section III describes the dataset. Section IV details the hybrid feature engineering and model training methodology. Section V describes the experimental setup, and Section VI presents results and analysis. Section VII describes the system architecture and deployment, Section VIII discusses findings and implications, and Section IX concludes.

II. RELATED WORK

Early work on automated phishing detection focused on engineered features drawn from message structure rather than raw text statistics. Chandrasekaran et al. proposed one of the first systems built specifically around structural properties of phishing emails, showing that features such as HTML structure, hyperlink counts, and script content could separate phishing from legitimate mail without deep semantic modeling [3], an approach conceptually close to the metadata channel used in this work. Fette et al. introduced PILFER, a machine-learning classifier trained on features designed to

highlight “user-targeted deception,” including IP-address-based links, mismatched link text and destination, HTML content, and embedded-link counts; their system correctly identified over 96% of phishing emails at a very low false-positive rate and remains a foundational reference for feature-based phishing email classification [4]. Basnet et al. compared several classical learning algorithms — support vector machines, neural networks, and others — on structural and content features of email, confirming that ensembles of simple engineered features could rival more complex pipelines [5]. Bergholz et al. extended this line of work with model-based features derived from statistical topic models of email content, improving on purely bag-of-words baselines and demonstrating that richer content modeling adds value beyond surface keyword counts [6].

On the website side of phishing, Zhang et al.'s CANTINA used TF-IDF — the same core text-representation technique adopted in this paper — to identify phishing web pages by comparing page content against search-engine-retrieved reference pages, and Xiang et al. later extended this into CANTINA+, a feature-rich framework combining HTML DOM analysis, search-engine signals, and third-party services with machine learning to substantially reduce false positives [7], [8]. While these systems target web pages rather than email bodies, they establish the effectiveness of TF-IDF as a lexical backbone, which this work carries over directly to the email domain.

More recent literature has moved toward hybrid and deep-learning approaches. Bountakas and Xenakis proposed HELPHED, a hybrid ensemble learning method that stacks lexical, structural, and semantic features across parallel classifiers, reporting accuracy above 99% on benchmark corpora and reinforcing the premise that no single feature family is sufficient on its own [9]. Champa et al. curated and benchmarked multiple phishing email datasets with systematic feature analysis, providing an empirical basis for feature selection in classifiers trained on Kaggle-style corpora similar to the one used here [10]. In parallel, transformer-based methods have shown strong results: fine-tuned BERT-style models have been reported to reach F1-scores near 0.99 on large phishing-email corpora, at the cost of substantially higher training and inference compute than classical TF-IDF pipelines [11]. Survey work by Dada et al. and by Gangavarapu et al. synthesizes this broader literature, cataloguing the progression from rule-based filters through classical machine learning to deep learning, and both identify

feature engineering — particularly the combination of content and structural features — as a recurring theme behind the strongest reported results [12], [13].

PhishGuard AI is positioned within this hybrid tradition: rather than relying on a single feature family or a compute-intensive transformer, it pairs a lightweight, interpretable 12-feature structural vector with a standard TF-IDF text representation, concatenated through sparse matrix operations into a single classical-ML-compatible feature space. This design keeps training and inference cost low enough for real-time use in a web application while still capturing the complementary signal that hybrid-feature studies such as HELPHED [9] and the structural-feature line of work from Chandrasekaran et al. [3] onward have shown to be valuable.

III. DATASET DESCRIPTION

The system is trained and evaluated on the Kaggle Phishing Email Dataset [14], a corpus of 18,650 labeled emails distributed across two columns: Email Text, containing the raw email body, and Email Type, a categorical label of either “Safe Email” or “Phishing Email.” Labels are numerically encoded during preprocessing (0 = Safe, 1 = Phishing) via a keyword-normalization routine that also tolerates alternate label spellings such as “spam” or “malicious.” The class distribution is imbalanced at approximately 60% Phishing and 40% Safe examples.

Because the imbalance is moderate rather than extreme, the pipeline does not apply resampling; instead, it relies on stratified splitting ($\text{test_size} = 0.20$, $\text{stratify} = y$) to hold the class ratio constant across an 80/20 train–test partition, and reports precision, recall, and F1-score alongside accuracy so that class-specific performance remains visible even though accuracy alone could be misleading under imbalance.

Email length and content vary considerably across the corpus: safe emails range from short transactional notices to lengthy newsletters and threaded correspondence, while phishing emails are frequently shorter and templated, built around a single call to action. A meaningful subset of phishing samples embed raw HTML markup and embedded hyperlinks — including some pointing to bare IP addresses rather than named domains — together with dense urgency or financial-incentive vocabulary, while safe emails more rarely exhibit these traits in combination. This asymmetry, present in

the raw text and independent of any single vocabulary term, is precisely what motivates the structural metadata channel described in Section IV: features such as `url_count`, `has_ip_url`, `urgent_word_count`, and `has_html_tags` are cheap to compute directly from the raw string and make this structural asymmetry explicit to the classifier rather than requiring it to be inferred indirectly through vocabulary alone.

IV. METHODOLOGY

A. Feature Engineering

PhishGuard AI represents each email through two complementary channels that are engineered separately and then fused.

1) Structural/Behavioral Metadata Features: Twelve numeric features are computed directly from the raw, uncleaned email string prior to any text normalization, because several signals — HTML tags, capitalization, raw IP-literal links — are destroyed by the lowercasing and tag-stripping the text channel requires. The twelve features fall into four conceptual groups, summarized with detection method and phishing rationale in Table I. Link-based features (`url_count`, `has_ip_url`) target the mechanism of a phishing attack directly, since nearly every phishing email exists to drive the reader toward a malicious link; an IP-literal URL is a particularly strong signal because legitimate organizations overwhelmingly link through named, brand-consistent domains. Punctuation and formatting features (`exclamation_count`, `question_count`, `uppercase_ratio`) operationalize the urgency-and-pressure register that social-engineering research consistently identifies as a hallmark of phishing pretexts. Lexical trigger features (`urgent_word_count`, `money_word_count`) count token matches, after punctuation-stripping, against two hand-built lexicons — a 38-term urgency/account-security lexicon (e.g., “verify,” “suspended,” “immediately,” “unauthorized”) and a 29-term financial-incentive lexicon (e.g., “prize,” “refund,” “bitcoin,” “lottery”). Length and composition features (`avg_word_len`, `log_char_count`, `log_word_count`, `digit_ratio`, `has_html_tags`) capture coarser statistical shape: character and word counts are log-scaled via $\log_{1p}$ to compress the influence of unusually long outlier emails, while `digit_ratio` and `has_html_tags` flag numeric-heavy content and raw markup respectively, both common in templated phishing kits but rare in ordinary plain-text correspondence.

TABLE I. THE 12 STRUCTURAL/BEHAVIORAL METADATA FEATURE

Feature	Detection Method / Formula	Phishing Signal Rationale
url_count	Regex count of http/https hyperlinks: <code>https?://\S+</code>	Phishing emails typically embed one or more links to spoofed credential-harvesting or redirect pages.
has_ip_url	Binary: any URL host is a literal IPv4 address, e.g. <code>https?://(\d{1,3}\.){3}\d{1,3}</code>	Legitimate organizations rarely link via raw IP; IP-literal URLs are a classic obfuscation tactic that evades domain-reputation checks.
exclamation_count	Raw count of “!” characters in the email body	Excessive exclamation punctuation manufactures urgency and pressures the reader toward an impulsive click.
question_count	Raw count of “?” characters in the email body	Rhetorical/interrogative framing (“Did you authorize this?”) is a common social-engineering hook in credential-theft lures.
uppercase_ratio	Fraction of whitespace-split tokens that are ALL-CAPS and length > 1	ALL-CAPS words (“URGENT,” “SUSPENDED”) are a visual urgency cue rarely used at scale in routine correspondence.
urgent_word_count	Count of punctuation-stripped tokens matching a 38-term urgency/account lexicon	Directly operationalizes the psychological pressure tactic central to phishing pretexts.
money_word_count	Count of punctuation-stripped tokens matching a 29-term financial-incentive lexicon	Captures the reward-bait pattern common to advance-fee, refund, and lottery-style scams.
avg_word_len	Mean character length of whitespace-split tokens	Phishing text trends toward short, simple, action-oriented vocabulary versus longer legitimate business prose.
log_char_count	$\log_{1p}(\text{total character count})$ of the raw email	Captures overall message length on a compressed scale; log-scaling limits outlier-length influence.
log_word_count	$\log_{1p}(\text{total whitespace-delimited word count})$	Companion length signal; phishing emails are frequently shorter and more templated than legitimate mail.
digit_ratio	Fraction of characters in the raw email that are digits	Elevated digit density can indicate fake reference/case numbers or monetary amounts used to lend false legitimacy.
has_html_tags	Binary: raw HTML tag pattern <code><[a-zA-Z][^>]*></code> found in text	Raw HTML (tracking pixels, hidden forms, styled buttons) in a plain-text body is a common phishing delivery mechanism.

2) TF-IDF Text Features: In parallel, the raw email text is cleaned — HTML stripped, URLs and email addresses replaced with placeholder tokens (“URL”, “EMAIL”) to

preserve their presence as a signal without leaking specific identifiers, lowercased, and reduced to alphabetic tokens — before being vectorized with scikit-learn's `TfidfVectorizer`

[15], configured with `max_features = 10,000`, `ngram_range = (1, 2)`, and `sublinear_tf = True`. Restricting the vocabulary to the 10,000 highest-scoring unigrams and bigrams keeps the resulting matrix tractable while still capturing short phrases (“click here,” “verify your,” “wire transfer”) that jointly carry more phishing signal than either constituent word alone; sublinear term-frequency scaling ($1 + \log(\text{tf})$) dampens the influence of a term that simply repeats many times within one email, preventing a single message that repeats “urgent” a dozen times from dominating its own TF-IDF weight vector.

3) Hybrid Fusion: The two channels are combined through sparse horizontal stacking. The metadata matrix is scaled to the $[0, 1]$ range with a `MinMaxScaler` fit only on the training partition (to prevent test-set leakage), converted to a sparse `csr_matrix` for format compatibility, and concatenated with the TF-IDF sparse matrix using SciPy's `sparse.hstack` routine [16], producing a single $(n_samples, 10,012)$ hybrid representation — 10,000 TF-IDF dimensions plus 12 metadata dimensions — passed to every hybrid-trained classifier as one input.

Pseudocode 1 summarizes the two-channel pipeline used both at training time and at inference time in the deployed application (Section VII):

```
INPUT: raw_email_text
-----
Channel 1 -- Structural Metadata
  meta_raw    <- extract_metadata(raw_email_text)
                                     # 12-dim,
dense
  meta_scaled <- MinMaxScaler.transform(meta_raw)
                                     # scaled to
[0,1]

Channel 2 -- Lexical / TF-IDF
  cleaned    <- clean_email(raw_email_text)
  tfidf_vec   <- TfidfVectorizer.transform(cleaned)
                                     # 10000-dim,
sparse

Fusion
  X_hybrid <- scipy.sparse.hstack(
                                     [tfidf_vec,
  csr_matrix(meta_scaled)])
                                     # (1,
10012), sparse

OUTPUT: label, probability <-
model.predict(X_hybrid)
-----
```

B. Model Training

Four classifiers are trained and compared to isolate the effect of the hybrid representation and identify the strongest production model. Complement Naive Bayes ($\alpha = 0.1$) is trained on the TF-IDF matrix alone, since the algorithm's probabilistic formulation is not well suited to a heterogeneously-scaled hybrid block; ComplementNB is used in preference to standard Multinomial Naive Bayes specifically because it corrects the multinomial model's known bias on imbalanced classes [17], which matters given the dataset's approximate 60/40 split. The remaining three classifiers are trained on the full 10,012-dimensional hybrid representation. Logistic Regression ($C = 1.0$, `lbfgs` solver, `max_iter = 1000`) serves as a linear baseline and, because its coefficients are directly interpretable, doubles as a diagnostic tool for identifying which terms and metadata features push a prediction toward “Phishing” versus “Safe.” Random Forest (200 trees) provides a non-linear, non-parametric baseline that additionally yields feature-importance rankings useful for validating whether the metadata features are actually used by the model rather than being dominated by the much larger TF-IDF block. Finally, a Multilayer Perceptron with three hidden layers (256, 128, 64 units; ReLU activations; Adam optimizer) is trained as the highest-capacity model compared, with early stopping (`validation_fraction = 0.1`, `n_iter_no_change = 5`, max 50 epochs) used to guard against overfitting given the network's larger parameter count relative to the linear and tree-based alternatives. All hybrid-trained models operate on identical training and test partitions (an 80/20 stratified split, `random_state = 42`) so that performance differences reflect model capacity and representation use rather than data variation; the MLP alone requires the sparse hybrid matrix to be densified (`.toarray()`) before fitting, since scikit-learn's `MLPClassifier` does not accept sparse input directly.

V. EXPERIMENTAL SETUP

All experiments were implemented in Python 3.13 using a standard open-source scientific computing stack. Feature engineering and sparse-matrix fusion rely on SciPy's sparse module for the `hstack` and `csr_matrix` operations that combine the TF-IDF and metadata channels [16], while NumPy and pandas handle numerical arrays and the tabular dataset respectively. All four classifiers, the `TfidfVectorizer`, the `MinMaxScaler`, and the evaluation metrics (accuracy, precision, recall, F1-score, confusion matrix, ROC/AUC) are



भारतीय संगणक एवं प्रौद्योगिकी संस्थान INDIAN INSTITUTE OF COMPUTING AND TECHNOLOGY

AFFILIATED: I-STEM, OFFICE OF THE PRINCIPAL SCIENTIFIC ADVISER TO THE GOVERNMENT OF INDIA

implemented via scikit-learn [15]. Visualization of the eight result plots described in Section VI uses Matplotlib and Seaborn. The dataset is partitioned once into an 80% training set and a 20% held-out test set using a stratified split with a fixed random seed (`random_state = 42`) to keep the class ratio constant across partitions and make results reproducible across runs. Model artifacts — the fitted TF-IDF vectorizer, the fitted MinMaxScaler, the trained best-performing model, and its metadata feature-name ordering — are serialized together into a single bundle (`models/best_model.pkl`) via Python's pickle

module, so that a single load restores the entire inference pipeline in the deployed application described in Section VII. The web interface is built with Streamlit, an open-source Python application framework [18].

VI. CODEBASE AND IT STRUCTURE

A. File Structure

```
Project 2/
├── app.py                ← Streamlit Web App → streamlit run app.py
├── phishing_email_detection.py ← Main ML Pipeline → python phishing_email_detection.py
├── metadata_features.py   ← 12-feature email metadata extractor
├── download_data.py       ← Auto-downloads dataset
├── train_all.py           ← Headless training script
├── test_inf.py            ← Quick inference test
├── data/
│   └── Phishing_Email.csv ← Created by download_data.py
├── models/
│   ├── best_model.pkl     ← Saved after training
│   └── results_summary.csv
└── plots/                 ← All generated charts (PNG)
```

B. `download_data.py`

```
"""
=====

Phishing Email Detection – Dataset Downloader
IICT Summer Internship Project 2
=====

Downloads the Phishing Email Dataset (Phishing_Email.csv)
from a verified public GitHub source and saves it to data/.
"""
```



भारतीय संगणक एवं प्रौद्योगिकी संस्थान INDIAN INSTITUTE OF COMPUTING AND TECHNOLOGY

AFFILIATED: I-STEM, OFFICE OF THE PRINCIPAL SCIENTIFIC ADVISER TO THE GOVERNMENT OF INDIA

```
import os
import urllib.request

# — Configuration

DATASET_URL = (
    "https://raw.githubusercontent.com/"
    "SINANFIROZ/Phishing-Email-Detector/master/Phishing_Email.csv"
)
DATA_DIR = os.path.join(os.path.dirname(__file__), "data")
SAVE_PATH = os.path.join(DATA_DIR, "Phishing_Email.csv")

HEADERS = {
    "User-Agent": (
        "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
        "AppleWebKit/537.36 (KHTML, like Gecko) "
        "Chrome/120.0.0.0 Safari/537.36"
    )
}

# — Main

def download() -> str:
    """Download dataset and return local path."""
    os.makedirs(DATA_DIR, exist_ok=True)

    if os.path.exists(SAVE_PATH):
        size_kb = os.path.getsize(SAVE_PATH) / 1024
        print(f"[INFO] Dataset already present: {SAVE_PATH} ({size_kb:.0f} KB)")
        return SAVE_PATH

    print(f"[INFO] Downloading dataset from:\n {DATASET_URL}")
    req = urllib.request.Request(DATASET_URL, headers=HEADERS)

    with urllib.request.urlopen(req, timeout=60) as response:
        total = int(response.headers.get("Content-Length", 0))
        downloaded = 0
```



भारतीय संगणक एवं प्रौद्योगिकी संस्थान INDIAN INSTITUTE OF COMPUTING AND TECHNOLOGY

AFFILIATED: I-STEM, OFFICE OF THE PRINCIPAL SCIENTIFIC ADVISER TO THE GOVERNMENT OF INDIA

```
chunk    = 8192
with open(SAVE_PATH, "wb") as f:
    while True:
        data = response.read(chunk)
        if not data:
            break
        f.write(data)
        downloaded += len(data)
        if total > 0:
            pct = min(downloaded / total * 100, 100)
            kb = downloaded / 1024
            print(f"\r Downloading ... {pct:5.1f}% ({kb:.0f} KB)", end="", flush=True)

print(f"\n[OK] Saved to: {SAVE_PATH}")
return SAVE_PATH

def verify(path: str) -> None:
    """Quick sanity check on the downloaded file."""
    import pandas as pd
    df = pd.read_csv(path)
    expected = {"Email Text", "Email Type"}
    missing = expected - set(df.columns)
    if missing:
        raise ValueError(f"Missing expected columns: {missing}")
    print(f"[OK] Rows : {len(df):,}")
    print(f"[OK] Columns: {df.columns.tolist()}")
    print(f"[OK] Labels : {df['Email Type'].unique().tolist()}")

if __name__ == "__main__":
    path = download()
    print("[INFO] Verifying dataset integrity ...")
    verify(path)
    print("\n[DONE] Dataset is ready for use.")
```



भारतीय संगणक एवं प्रौद्योगिकी संस्थान INDIAN INSTITUTE OF COMPUTING AND TECHNOLOGY

AFFILIATED: I-STEM, OFFICE OF THE PRINCIPAL SCIENTIFIC ADVISER TO THE GOVERNMENT OF INDIA

C. metadata_features.py

```
"""
=====
Phishing Email Detection – Metadata Feature Extractor
IICT Summer Internship Project 2
=====

Extracts 12 structural/behavioural features from raw email text:

1. url_count      – number of http/https links
2. has_ip_url     – link contains raw IP address
3. exclamation_count – number of "!" characters
4. uppercase_ratio – fraction of UPPERCASE words
5. urgent_word_count – count of urgency trigger words
6. money_word_count – count of financial trigger words
7. avg_word_len   – average token length
8. char_count     – total character count (log-scaled)
9. word_count     – total word count (log-scaled)
10. digit_ratio   – fraction of digit characters
11. special_char_count – non-alphanumeric, non-space chars
12. has_html_tags – presence of <a>, <img>, etc.
"""

import re
import math
import numpy as np
from typing import List, Dict, Union

# — Trigger word lists —
URGENT_WORDS = {
    "urgent", "immediately", "action", "required", "suspended", "verify",
    "confirm", "validate", "update", "account", "expire", "expires",
    "limited", "warning", "alert", "important", "attention", "critical",
    "deadline", "asap", "now", "today", "failure", "compromised",
    "unusual", "unauthorised", "unauthorized", "detected", "activity",
    "suspended", "disabled", "locked", "click", "here", "login",
    "sign in", "access", "restore",
}
```

```
MONEY_WORDS = {
    "free", "prize", "win", "winner", "cash", "bonus", "reward",
    "offer", "deal", "discount", "sale", "earn", "income", "invest",
    "profit", "million", "thousand", "dollar", "usd", "bitcoin",
    "crypto", "refund", "claim", "inheritance", "lottery",
}

# — Compiled regex patterns —
_URL_PATTERN = re.compile(r"https?://S+", re.IGNORECASE)
_IP_IN_URL = re.compile(r"https?://\d{1,3}(\.\d{1,3}){3}", re.IGNORECASE)
_HTML_TAG = re.compile(r"<[a-zA-Z][^>]*>")
_DIGIT_RE = re.compile(r"\d")
_SPECIAL_CHAR = re.compile(r"^[a-zA-Z0-9\s]")

class EmailMetadataExtractor:
    """Extract a fixed-length feature vector from a raw email string."""

    FEATURE_NAMES: List[str] = [
        "url_count",
        "has_ip_url",
        "exclamation_count",
        "uppercase_ratio",
        "urgent_word_count",
        "money_word_count",
        "avg_word_len",
        "char_count",
        "word_count",
        "digit_ratio",
        "special_char_count",
        "has_html_tags",
    ]

    def extract(self, text: str) -> Dict[str, float]:
        """Return a dict of 12 numeric features for one email string."""
        if not isinstance(text, str):
            text = str(text) if text else ""
```



भारतीय संगणक एवं प्रौद्योगिकी संस्थान
INDIAN INSTITUTE OF COMPUTING AND TECHNOLOGY

AFFILIATED: I-STEM, OFFICE OF THE PRINCIPAL SCIENTIFIC ADVISER TO THE GOVERNMENT OF INDIA

```
text_lower = text.lower()
words     = text.split()
n_words   = len(words)
n_chars   = len(text)

urls = _URL_PATTERN.findall(text)
url_count = len(urls)

has_ip_url = float(bool(_IP_IN_URL.search(text)))
exclamation_count = float(text.count("!"))

if n_words > 0:
    upper_words = sum(1 for w in words if w.isupper() and len(w) > 1)
    uppercase_ratio = upper_words / n_words
else:
    uppercase_ratio = 0.0

words_lower = text_lower.split()
urgent_word_count = float(
    sum(1 for w in words_lower if w.strip(".,!?:\"'()") in URGENT_WORDS)
)

money_word_count = float(
    sum(1 for w in words_lower if w.strip(".,!?:\"'()") in MONEY_WORDS)
)

avg_word_len = sum(len(w) for w in words) / max(n_words, 1)
char_count = float(math.log1p(n_chars))
word_count = float(math.log1p(n_words))

n_digits = len(_DIGIT_RE.findall(text))
digit_ratio = n_digits / max(n_chars, 1)

special_char_count = float(len(_SPECIAL_CHAR.findall(text)))
has_html_tags = float(bool(_HTML_TAG.search(text)))

return {
    "url_count" : float(url_count),
```



भारतीय संगणक एवं प्रौद्योगिकी संस्थान INDIAN INSTITUTE OF COMPUTING AND TECHNOLOGY

AFFILIATED: I-STEM, OFFICE OF THE PRINCIPAL SCIENTIFIC ADVISER TO THE GOVERNMENT OF INDIA

```
"has_ip_url"      : has_ip_url,
"exclamation_count" : exclamation_count,
"uppercase_ratio"  : uppercase_ratio,
"urgent_word_count" : urgent_word_count,
"money_word_count" : money_word_count,
"avg_word_len"    : avg_word_len,
"char_count"      : char_count,
"word_count"      : word_count,
"digit_ratio"     : digit_ratio,
"special_char_count" : special_char_count,
"has_html_tags"   : has_html_tags,
}

def extract_vector(self, text: str) -> np.ndarray:
    """Return a 1D numpy array in the order of FEATURE_NAMES."""
    d = self.extract(text)
    return np.array([d[k] for k in self.FEATURE_NAMES], dtype=np.float64)

def extract_metadata_matrix(texts: List[str],
                             extractor: EmailMetadataExtractor = None
                             ) -> np.ndarray:
    """Process a list of email strings → 2D numpy matrix (n_emails, n_features)."""
    if extractor is None:
        extractor = EmailMetadataExtractor()
    return np.vstack([extractor.extract_vector(t) for t in texts])

if __name__ == "__main__":
    samples = [
        "URGENT!!! Your account has been SUSPENDED. Click http://192.168.1.1/verify NOW!",
        "Hi John, please find the meeting notes attached. Best regards, Sarah.",
        "WIN a FREE iPhone! You are the lucky WINNER. Claim NOW at http://scam.xyz !!!",
        "Dear customer, your invoice for order #12345 is attached. No action required.",
    ]
    ext = EmailMetadataExtractor()
    print(f'{"Email (truncated)":<55} {"URL":>4} {"Excl":>5} {"Urgnt":>6} {"UpRt":>5}')}
    print("-" * 80)
```



भारतीय संगणक एवं प्रौद्योगिकी संस्थान INDIAN INSTITUTE OF COMPUTING AND TECHNOLOGY

AFFILIATED: I-STEM, OFFICE OF THE PRINCIPAL SCIENTIFIC ADVISER TO THE GOVERNMENT OF INDIA

```
for s in samples:
    feat = ext.extract(s)
    print(
        f"{s[:53]:<55} "
        f"{feat['url_count']:>4.0f} "
        f"{feat['exclamation_count']:>5.0f} "
        f"{feat['urgent_word_count']:>6.0f} "
        f"{feat['uppercase_ratio']:>5.3f}"
    )
```

D. phishing_email_detection.py

```
"""
=====
|| PHISHING EMAIL DETECTION – IICT Internship Project 2 ||
|| Complete ML Pipeline: EDA → Preprocessing → Training → Evaluation ||
=====
||
|| Models : Naive Bayes | Logistic Regression | Random Forest | MLP ||
|| Features : TF-IDF (text) + 12 hand-crafted structural metadata features ||
|| Dataset : Phishing Email Dataset (~18,650 emails – text, label) ||
=====
=====

```

HOW TO RUN LOCALLY:

```
python phishing_email_detection.py
(Expects: data/Phishing_Email.csv — run download_data.py first)
```

```
"""

import os, re, sys, math, warnings, pickle
from collections import Counter
from typing import List, Dict, Optional, Tuple

import numpy as np
import pandas as pd
import scipy.sparse as sp
```



भारतीय संगणक एवं प्रौद्योगिकी संस्थान
INDIAN INSTITUTE OF COMPUTING AND TECHNOLOGY

AFFILIATED: I-STEM, OFFICE OF THE PRINCIPAL SCIENTIFIC ADVISER TO THE GOVERNMENT OF INDIA

```
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import ComplementNB
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, confusion_matrix, classification_report,
    roc_curve, auc,
)

warnings.filterwarnings("ignore")
os.makedirs("plots", exist_ok=True)
os.makedirs("models", exist_ok=True)

# — Dark cybersecurity theme —
plt.rcParams.update({
    "figure.facecolor": "#0d0d1a", "axes.facecolor": "#111127",
    "axes.edgecolor": "#1a1a3e", "text.color": "#e8e8ff",
    "axes.labelcolor": "#e8e8ff", "xtick.color": "#e8e8ff",
    "ytick.color": "#e8e8ff", "axes.titlecolor": "#00d4ff",
    "axes.grid": True, "grid.color": "#1a1a3e", "grid.alpha": 0.4,
    "font.size": 11,
})
CYAN="#00d4ff"; ORANGE="#ff6b35"; GREEN="#39ff14"; PURPLE="#b44be1"
COLORS = [CYAN, ORANGE, GREEN, PURPLE]

#

print("\n" + "="*70)
print(" SECTION 1 — Dataset Loading")
```



भारतीय संगणक एवं प्रौद्योगिकी संस्थान
INDIAN INSTITUTE OF COMPUTING AND TECHNOLOGY

AFFILIATED: I-STEM, OFFICE OF THE PRINCIPAL SCIENTIFIC ADVISER TO THE GOVERNMENT OF INDIA

```
print("="*70)

def find_csv() -> str:
    """Auto-detect the Phishing Email CSV on Kaggle or locally."""
    kaggle_root = "/kaggle/input"
    if os.path.isdir(kaggle_root):
        for root, _, files in os.walk(kaggle_root):
            for fname in sorted(files):
                if not fname.endswith(".csv"):
                    continue
                path = os.path.join(root, fname)
                try:
                    peek = pd.read_csv(path, nrows=3)
                    cols = [c.lower().strip() for c in peek.columns]
                    if any("email" in c and "text" in c for c in cols) or \
                        any("email" in c and "type" in c for c in cols):
                        print(f" [Kaggle] Dataset found: {path}")
                        return path
                    if any("text" in c for c in cols) and \
                        any(c in cols for c in ["label", "type", "class", "target"]):
                        print(f" [Kaggle] Dataset found (generic): {path}")
                        return path
                except Exception:
                    continue
    candidates = [
        os.path.join(os.path.dirname(os.path.abspath(__file__)), "data", "Phishing_Email.csv"),
        os.path.join(os.path.dirname(os.path.abspath(__file__)), "data", "phishing_email.csv"),
    ]
    for path in candidates:
        if os.path.exists(path):
            print(f" [Local] Dataset found: {path}")
            return path
    raise FileNotFoundError(
        "\n Dataset not found!\n"
        " Local: run python download_data.py first"
    )

CSV_PATH = find_csv()
```



भारतीय संगणक एवं प्रौद्योगिकी संस्थान
INDIAN INSTITUTE OF COMPUTING AND TECHNOLOGY

AFFILIATED: I-STEM, OFFICE OF THE PRINCIPAL SCIENTIFIC ADVISER TO THE GOVERNMENT OF INDIA

```
df_raw = pd.read_csv(CSV_PATH)
df_raw.columns = [c.strip() for c in df_raw.columns]
df_raw.drop(columns=[c for c in df_raw.columns if "unnamed" in c.lower()], inplace=True)
print(f"\n Raw shape : {df_raw.shape}")
print(f" Columns : {df_raw.columns.tolist()}")
```

```
TEXT_COL = next((c for c in df_raw.columns if "text" in c.lower()), df_raw.columns[0])
LABEL_COL = next((c for c in df_raw.columns if "type" in c.lower()
                  or "label" in c.lower() or "class" in c.lower()), df_raw.columns[-1])
print(f"\n Using text column : '{TEXT_COL}'")
print(f" Using label column : '{LABEL_COL}'")
```

```
#
```

```
print("\n" + "="*70)
print(" SECTION 2 — Data Cleaning & EDA")
print("="*70)
```

```
df = df_raw[[TEXT_COL, LABEL_COL]].copy()
df.rename(columns={TEXT_COL: "text", LABEL_COL: "label"}, inplace=True)
df["text"] = df["text"].fillna("").astype(str)
df["label"] = df["label"].fillna("").astype(str).strip()
df.dropna(subset=["label"], inplace=True)
df = df[df["label"] != ""]
```

```
def encode_label(val: str) -> int:
    v = val.lower()
    if "phish" in v or v in ("1", "spam", "malicious", "fake"):
        return 1
    return 0
```

```
df["label_int"] = df["label"].apply(encode_label)
df["word_count"] = df["text"].str.split().str.len().fillna(0).astype(int)
print(f"\n Clean shape : {df.shape}")
print(f" Class counts: {df['label'].value_counts().to_dict()}")
```

```
# EDA plots
```

```
counts = df["label"].value_counts()
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
fig.suptitle("EDA ► Class Distribution: Phishing vs Safe Email", fontsize=14, color=CYAN, fontweight="bold")
bars = axes[0].bar(counts.index, counts.values, color=[GREEN, ORANGE], edgecolor="white", lw=0.8)
for bar, val in zip(bars, counts.values):
    axes[0].text(bar.get_x()+bar.get_width()/2, bar.get_height()+counts.max()*0.01,
        f"{val:},", ha="center", va="bottom", fontsize=12)
axes[0].set_ylabel("Count"); axes[0].set_title("Email Count by Class")
axes[1].pie(counts.values, labels=counts.index, colors=[GREEN, ORANGE],
    autopct="%1.1f%%", startangle=90, textprops={"color": "white"})
axes[1].set_title("Class Balance")
plt.tight_layout(); plt.savefig("plots/01_class_distribution.png", dpi=150, bbox_inches="tight"); plt.close()
print(" [Plot saved] → plots/01_class_distribution.png")
```

```
fig, ax = plt.subplots(figsize=(13, 5))
for lname, color, key in [("Safe Email", GREEN, 0), ("Phishing Email", ORANGE, 1)]:
    ax.hist(df[df["label_int"]==key]["word_count"].clip(0,1000), bins=70, alpha=0.72, color=color, label=lname)
ax.set_xlabel("Word Count"); ax.set_ylabel("Frequency"); ax.legend()
ax.set_title("EDA ► Email Word Count by Class")
plt.tight_layout(); plt.savefig("plots/02_email_length.png", dpi=150, bbox_inches="tight"); plt.close()
print(" [Plot saved] → plots/02_email_length.png")
```

#

```
print("\n" + "="*70)
print(" SECTION 3 — Structural Metadata Feature Engineering (12 features)")
print("="*70)
```

```
URGENT_WORDS = frozenset({
    "urgent", "immediately", "action", "required", "suspended", "verify", "confirm",
    "validate", "update", "account", "expire", "expires", "limited", "warning", "alert",
    "important", "attention", "critical", "deadline", "asap", "now", "today", "failure",
    "compromised", "unusual", "unauthorized", "detected", "activity", "disabled",
    "locked", "click", "here", "login", "signin", "access", "restore", "security",
    "password", "credential", "suspended", "deactivated", "reactivate",
})
MONEY_WORDS = frozenset({
```

```
"free","prize","win","winner","cash","bonus","reward","offer","deal",
"discount","sale","earn","income","invest","profit","million","thousand",
"dollar","usd","bitcoin","crypto","refund","claim","inheritance","lottery",
"congratulations","selected","chosen","gift","voucher","coupon",
})
```

```
_URL_RE = re.compile(r"https?://\S+", re.I)
_IP_URL_RE = re.compile(r"https?://(\d{1,3}\.){3}\d{1,3}", re.I)
_HTML_RE = re.compile(r"<[a-zA-Z][^>]*>")
_DIGIT_RE = re.compile(r"\d")
```

```
META_FEATURE_NAMES: List[str] = [
    "url_count",      # 1 – number of URLs
    "has_ip_url",     # 2 – any URL with raw IP address
    "exclamation_count", # 3 – count of "!"
    "question_count",  # 4 – count of "?"
    "uppercase_ratio", # 5 – fraction of fully-uppercase words
    "urgent_word_count", # 6 – urgency trigger words
    "money_word_count", # 7 – financial trigger words
    "avg_word_len",    # 8 – mean token length
    "log_char_count",  # 9 – log(1 + total char count)
    "log_word_count",  # 10 – log(1 + total word count)
    "digit_ratio",     # 11 – fraction of digit characters
    "has_html_tags",   # 12 – presence of HTML markup
]
```

```
def extract_metadata(text: str) -> np.ndarray:
    """Return a 1D numpy array of 12 numeric features for one email."""
    if not isinstance(text, str):
        text = str(text) if text else ""
    tl = text.lower(); words = text.split(); wl = tl.split(); nc = len(text)
    urls = _URL_RE.findall(text)
    feat = [
        float(len(urls)),
        float(bool(_IP_URL_RE.search(text))),
        float(text.count("!")),
        float(text.count("?")),
        sum(1 for w in words if w.isupper() and len(w)>1) / max(len(words),1),
```



भारतीय संगणक एवं प्रौद्योगिकी संस्थान INDIAN INSTITUTE OF COMPUTING AND TECHNOLOGY

AFFILIATED: I-STEM, OFFICE OF THE PRINCIPAL SCIENTIFIC ADVISER TO THE GOVERNMENT OF INDIA

```
float(sum(1 for w in wl if re.sub(r"[.,!?:\\"'()]", "", w) in URGENT_WORDS)),
float(sum(1 for w in wl if re.sub(r"[.,!?:\\"'()]", "", w) in MONEY_WORDS)),
sum(len(w) for w in words) / max(len(words), 1),
math.log1p(nc),
math.log1p(len(words)),
len(_DIGIT_RE.findall(text)) / max(nc, 1),
float(bool(_HTML_RE.search(text))),
]
return np.array(feats, dtype=np.float64)

def extract_metadata_batch(texts: List[str]) -> np.ndarray:
    return np.vstack([extract_metadata(t) for t in texts])

# Demo
demo_emails = [
    "URGENT: Your account SUSPENDED! Click http://192.0.0.1/verify NOW!!!",
    "Hi John, please find the attached quarterly report. Best, Sarah.",
    "You WON a FREE iPhone!!! Claim NOW -> http://prize.xyz/claim !!!!!!",
]
print("\n Metadata demo:")
for e in demo_emails:
    feat = extract_metadata(e)
    print(f" URL={feat[0]:.0f} Excl={feat[2]:.0f} Urgent={feat[5]:.0f} | {e[:60]}")

#
=====

print("\n" + "="*70)
print(" SECTION 4 — Email Text Preprocessing")
print("="*70)

_HTML_STRIP = re.compile(r"<[^\>]+>")
_URL_STRIP = re.compile(r"https?://\S+", re.I)
_EMAIL_STRIP = re.compile(r"\S+@\S+")
_NON_ALPHA = re.compile(r"^[^a-z\s]")
_SPACES = re.compile(r"\s+")

def clean_email(text: str) -> str:
```



भारतीय संगणक एवं प्रौद्योगिकी संस्थान INDIAN INSTITUTE OF COMPUTING AND TECHNOLOGY

AFFILIATED: I-STEM, OFFICE OF THE PRINCIPAL SCIENTIFIC ADVISER TO THE GOVERNMENT OF INDIA

```
"""Replace URLs/emails with tokens → strip HTML → lowercase → normalise."""
if not isinstance(text, str):
    text = str(text) if text else ""
text = _HTML_STRIP.sub(" ", text)
text = _URL_STRIP.sub(" URL ", text)
text = _EMAIL_STRIP.sub(" EMAIL ", text)
text = text.lower()
text = _NON_ALPHA.sub(" ", text)
return _SPACES.sub(" ", text).strip()

print(" Cleaning all emails ...", end=" ", flush=True)
df["cleaned"] = df["text"].apply(clean_email)
print("Done ✓")

#

=====

print("\n" + "="*70)
print(" SECTION 5 — Feature Extraction: TF-IDF + 12 Metadata Features")
print("="*70)

X_text = df["cleaned"].values
X_raw = df["text"].values
y = df["label_int"].values

(X_text_tr, X_text_te,
 X_raw_tr, X_raw_te,
 y_train, y_test) = train_test_split(
    X_text, X_raw, y, test_size=0.20, random_state=42, stratify=y)
print(f"\n Train : {len(X_text_tr):,} | Test : {len(X_text_te):,}")

print(" Building TF-IDF ...", end=" ", flush=True)
tfidf = TfidfVectorizer(max_features=10_000, ngram_range=(1,2), sublinear_tf=True)
X_train_tfidf = tfidf.fit_transform(X_text_tr)
X_test_tfidf = tfidf.transform(X_text_te)
print("Done ✓")

print(" Extracting metadata ...", end=" ", flush=True)
```

```
M_train_raw = extract_metadata_batch(list(X_raw_tr))
M_test_raw = extract_metadata_batch(list(X_raw_te))
scaler = MinMaxScaler()
M_train = scaler.fit_transform(M_train_raw)
M_test = scaler.transform(M_test_raw)
print("Done ✓")

X_train_full = sp.hstack([X_train_tfidf, sp.csr_matrix(M_train)])
X_test_full = sp.hstack([X_test_tfidf, sp.csr_matrix(M_test)])
print(f" Combined matrix : {X_train_full.shape}")

#

=====

print("\n" + "="*70)
print(" SECTION 6 — Model Training (4 classifiers)")
print("="*70)

def train_eval(name: str, model, X_tr, X_te, y_tr, y_te, dense: bool = False):
    print(f"\n ► [{name}]", end=" ", flush=True)
    if dense:
        X_tr = X_tr.toarray() if sp.issparse(X_tr) else X_tr
        X_te = X_te.toarray() if sp.issparse(X_te) else X_te
    model.fit(X_tr, y_tr)
    y_pred = model.predict(X_te)
    met = {
        "Model" : name,
        "Accuracy" : accuracy_score(y_te, y_pred),
        "Precision": precision_score(y_te, y_pred, zero_division=0),
        "Recall" : recall_score(y_te, y_pred, zero_division=0),
        "F1-Score" : f1_score(y_te, y_pred, zero_division=0),
    }
    print(f"Acc={met['Accuracy']:.4f} F1={met['F1-Score']:.4f}")
    print(classification_report(y_te, y_pred, target_names=["Safe", "Phishing"], zero_division=0))
    return model, y_pred, met

all_results: List[dict] = []
trained_mdls: Dict[str, object] = {}
```

```
all_preds: Dict[str, np.ndarray] = {}

print(" — 1/4 Complement Naive Bayes (TF-IDF only)")
m, p, met = train_eval("Naive Bayes", ComplementNB(alpha=0.1),
                        X_train_tfidf, X_test_tfidf, y_train, y_test)
trained_mdls["NB"]=m; all_preds["NB"]=p; all_results.append(met)

print(" — 2/4 Logistic Regression (TF-IDF + Metadata)")
m, p, met = train_eval("Logistic Regression", LogisticRegression(C=1.0, max_iter=1000, n_jobs=-1),
                        X_train_full, X_test_full, y_train, y_test)
trained_mdls["LR"]=m; all_preds["LR"]=p; all_results.append(met)

print(" — 3/4 Random Forest (TF-IDF + Metadata)")
m, p, met = train_eval("Random Forest", RandomForestClassifier(n_estimators=200, n_jobs=-1, random_state=42),
                        X_train_full, X_test_full, y_train, y_test)
trained_mdls["RF"]=m; all_preds["RF"]=p; all_results.append(met)

print(" — 4/4 MLP Neural Network (TF-IDF + Metadata)")
m, p, met = train_eval("MLP Neural Network",
                        MLPClassifier(hidden_layer_sizes=(256,128,64), activation="relu",
                                      solver="adam", max_iter=50, random_state=42,
                                      early_stopping=True, validation_fraction=0.1, n_iter_no_change=5),
                        X_train_full, X_test_full, y_train, y_test, dense=True)
trained_mdls["MLP"]=m; all_preds["MLP"]=p; all_results.append(met)

#

=====

print("\n" + "="*70)
print(" SECTION 7 — Evaluation & Visualisation")
print("="*70)

results_df = pd.DataFrame(all_results).set_index("Model").round(4)
print("\n ═══ MODEL COMPARISON TABLE ═══")
print(results_df.to_string())

model_order = list(results_df.index)
KEY_MAP = {"Naive Bayes": "NB", "Logistic Regression": "LR",
```

```

"Random Forest": "RF", "MLP Neural Network": "MLP"}

metrics_cols = ["Accuracy", "Precision", "Recall", "F1-Score"]
x = np.arange(len(metrics_cols)); w = 0.18

fig, ax = plt.subplots(figsize=(14,6))
ax.set_title("Phishing Email Detection — Model Comparison", fontsize=13, pad=12)
for i, (name, color) in enumerate(zip(model_order, COLORS)):
    vals = results_df.loc[name, metrics_cols].values
    bars = ax.bar(x+i*w, vals, w, label=name, color=color, alpha=0.85)
    for bar, val in zip(bars, vals):
        ax.text(bar.get_x()+bar.get_width()/2, bar.get_height()+0.003,
                f'{val:.3f}', ha="center", va="bottom", fontsize=8)
ax.set_xticks(x+w*1.5); ax.set_xticklabels(metrics_cols)
ax.set_ylim(0,1.15); ax.set_ylabel("Score"); ax.legend()
plt.tight_layout(); plt.savefig("plots/04_model_comparison.png", dpi=150, bbox_inches="tight"); plt.close()
print(" [Plot saved] → plots/04_model_comparison.png")

fig, axes = plt.subplots(2,2, figsize=(13,10))
fig.suptitle("Confusion Matrices — All 4 Models", fontsize=14, color=CYAN)
for ax, name in zip(axes.flatten(), model_order):
    key = KEY_MAP[name]
    cm = confusion_matrix(y_test, all_preds[key])
    sns.heatmap(cm, annot=True, fmt="d", ax=ax, cmap="Blues", linewidths=0.6,
                xticklabels=["Safe", "Phishing"], yticklabels=["Safe", "Phishing"],
                annot_kws={"size":14, "weight":"bold"})
    ax.set_title(f'{name}\nAcc={results_df.loc[name, "Accuracy"]} | F1={results_df.loc[name, "F1-Score"]}')
    ax.set_xlabel("Predicted"); ax.set_ylabel("True")
plt.tight_layout(); plt.savefig("plots/05_confusion_matrices.png", dpi=150, bbox_inches="tight"); plt.close()
print(" [Plot saved] → plots/05_confusion_matrices.png")

fig, ax = plt.subplots(figsize=(9,7))
ax.set_title("ROC Curves — All 4 Models", fontsize=13)
for name, color in zip(model_order, COLORS):
    key = KEY_MAP[name]
    mod = trained_mdls[key]
    Xte = X_test_tfidf if key == "NB" else X_test_full
    if hasattr(mod, "predict_proba"):

```

```

proba = mod.predict_proba(Xte.toarray() if key=="MLP" and sp.issparse(Xte) else Xte)[: ,1]
else:
    proba = all_preds[key].astype(float)
fpr, tpr, _ = roc_curve(y_test, proba)
ax.plot(fpr, tpr, lw=2.5, color=color, label=f'{name} (AUC={auc(fpr,tpr):.3f})')
ax.plot([0,1],[0,1],"k--",lw=1.2,alpha=0.5)
ax.set_xlabel("FPR"); ax.set_ylabel("TPR"); ax.legend(loc="lower right")
plt.tight_layout(); plt.savefig("plots/06_roc_curves.png", dpi=150, bbox_inches="tight"); plt.close()
print(" [Plot saved] → plots/06_roc_curves.png")

tfidf_feature_names = list(tfidf.get_feature_names_out())
all_feature_names = tfidf_feature_names + META_FEATURE_NAMES
importances = trained_mdls["RF"].feature_importances_
top_idx = importances.argsort()[-20:][::-1]
top_names = [all_feature_names[i] if i < len(all_feature_names) else f'feat_{i}' for i in top_idx]
fig, ax = plt.subplots(figsize=(12,7))
ax.barh(list(reversed(top_names)), list(reversed(importances[top_idx])), color=CYAN)
ax.set_title("Random Forest — Top 20 Feature Importances"); ax.set_xlabel("Gini Importance")
plt.tight_layout(); plt.savefig("plots/07_feature_importance.png", dpi=150, bbox_inches="tight"); plt.close()
print(" [Plot saved] → plots/07_feature_importance.png")

#
=====

print("\n" + "="*70)
print(" SECTION 8 — Discussion & Key Findings")
print("="*70)
best_name = results_df["F1-Score"].idxmax()
print(f"""
Key phishing signals detected by the model:
• urgent_word_count — phishing emails use urgency language heavily
• exclamation_count — multiple "!" is a pressure tactic
• has_ip_url — raw IP links never appear in legitimate emails
• uppercase_ratio — ALL CAPS is a manipulation hallmark
• url_count — excessive links indicate phishing

★ Best model by F1-Score : {best_name}
★ F1-Score : {results_df.loc[best_name,"F1-Score"]:.4f}

```



भारतीय संगणक एवं प्रौद्योगिकी संस्थान INDIAN INSTITUTE OF COMPUTING AND TECHNOLOGY

AFFILIATED: I-STEM, OFFICE OF THE PRINCIPAL SCIENTIFIC ADVISER TO THE GOVERNMENT OF INDIA

```
★ Recall (security KPI) : {results_df.loc[best_name,"Recall"]:.4f}
""")

#

=====

print("="*70)
print(" SECTION 9 — Saving Best Model")
print("="*70)

best_key = KEY_MAP[best_name]
bundle = {
    "vectorizer" : tfidf,
    "scaler"      : scaler,
    "model"       : trained_mdls[best_key],
    "model_name"  : best_name,
    "meta_features": META_FEATURE_NAMES,
    "results"     : results_df.to_dict(),
}
with open("models/best_model.pkl", "wb") as f:
    pickle.dump(bundle, f)
print(f"\n ★ Best model : {best_name}")
print(f" ★ Saved to : models/best_model.pkl")
results_df.to_csv("models/results_summary.csv")
print(" ★ Results CSV: models/results_summary.csv")

# Quick inference demo
print("\n — Quick Inference Demo —")
test_emails = [
    "URGENT: Your PayPal account has been SUSPENDED! Verify at http://192.0.0.1/secure",
    "Hi, just confirming our 10am meeting tomorrow. See you then! - Mike",
    "WIN a FREE MacBook! You've been SELECTED! Claim at http://reward.xyz",
    "Please find the attached invoice for last month. Let me know if you have questions.",
]
for email in test_emails:
    cleaned_e = clean_email(email)
    tfidf_vec = tfidf.transform([cleaned_e])
    meta_vec = scaler.transform(extract_metadata(email).reshape(1,-1))
```



भारतीय संगणक एवं प्रौद्योगिकी संस्थान INDIAN INSTITUTE OF COMPUTING AND TECHNOLOGY

AFFILIATED: I-STEM, OFFICE OF THE PRINCIPAL SCIENTIFIC ADVISER TO THE GOVERNMENT OF INDIA

```
X_input = tfidf_vec if best_key=="NB" else sp.hstack([tfidf_vec, sp.csr_matrix(meta_vec)])
pred = trained_mdls[best_key].predict(X_input.toarray() if best_key=="MLP" else X_input)[0]
label = "🔴 PHISHING" if pred==1 else "🟢 SAFE"
print(f" {label} > {email[:68]}")

print("\n" + "="*70)
print(" ❧ Project 2 Complete! → outputs in plots/ and models/")
print("="*70 + "\n")
```

E. train_all.py

```
"""
=====

Phishing Email Detection – Full Training Pipeline
IICT Summer Internship Project 2
=====

Trains all 4 models headlessly and saves outputs.

Usage: python train_all.py
"""

import os, sys, re, pickle, warnings, math
import numpy as np
import pandas as pd
import scipy.sparse as sp
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt
import seaborn as sns

warnings.filterwarnings("ignore")
sys.path.insert(0, os.path.dirname(__file__))

from metadata_features import extract_metadata_matrix, EmailMetadataExtractor
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import ComplementNB
from sklearn.linear_model import LogisticRegression
```



भारतीय संगणक एवं प्रौद्योगिकी संस्थान INDIAN INSTITUTE OF COMPUTING AND TECHNOLOGY

AFFILIATED: I-STEM, OFFICE OF THE PRINCIPAL SCIENTIFIC ADVISER TO THE GOVERNMENT OF INDIA

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, roc_curve, auc
)

# — Style

plt.rcParams.update({
    "figure.facecolor": "#0d0d1a", "axes.facecolor": "#111127",
    "axes.edgecolor": "#1a1a3e", "text.color": "#e8e8ff",
    "axes.labelcolor": "#e8e8ff", "xtick.color": "#e8e8ff",
    "ytick.color": "#e8e8ff", "axes.titlecolor": "#00d4ff",
    "axes.grid": True, "grid.color": "#1a1a3e", "grid.alpha": 0.4,
})
CYAN="#00d4ff"; ORANGE="#ff6b35"; GREEN="#39ff14"; PURPLE="#b44be1"

os.makedirs("plots", exist_ok=True)
os.makedirs("models", exist_ok=True)

#

=====
print("\n" + "="*65)
print(" STEP 1 – Loading Dataset")
print("="*65)

DATA_PATH = os.path.join(os.path.dirname(__file__), "data", "Phishing_Email.csv")
if not os.path.exists(DATA_PATH):
    print("[ERROR] Dataset not found. Run download_data.py first.")
    sys.exit(1)

df = pd.read_csv(DATA_PATH)
df.rename(columns={"Email Text": "text", "Email Type": "label"}, inplace=True)
df.drop(columns=[c for c in df.columns if "Unnamed" in c], inplace=True)
df.dropna(subset=["text"], inplace=True)
```



भारतीय संगणक एवं प्रौद्योगिकी संस्थान
INDIAN INSTITUTE OF COMPUTING AND TECHNOLOGY

AFFILIATED: I-STEM, OFFICE OF THE PRINCIPAL SCIENTIFIC ADVISER TO THE GOVERNMENT OF INDIA

```
df["text"] = df["text"].astype(str)
df["label_int"] = (df["label"] == "Phishing Email").astype(int)
print(f" Rows : {len(df):,}")
print(f" Labels : {df['label'].value_counts().to_dict()}")

# EDA plot
counts = df["label"].value_counts()
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
fig.suptitle("Class Distribution – Phishing vs. Safe Email", fontsize=14, color=CYAN)
axes[0].bar(counts.index, counts.values, color=[GREEN, ORANGE], edgecolor="white", lw=0.6)
axes[0].set_title("Email Count by Class")
axes[1].pie(counts.values, labels=counts.index, colors=[GREEN, ORANGE],
            autopct="%1.1f%%", startangle=90, textprops={"color": "white"})
plt.tight_layout(); plt.savefig("plots/class_distribution.png", dpi=150, bbox_inches="tight"); plt.close()

#

=====

print("\n STEP 2 – Preprocessing & Feature Engineering")

def clean_email(t: str) -> str:
    t = re.sub(r"https?://\S+", " URL ", t)
    t = re.sub(r"\S+@\S+", " EMAIL ", t)
    t = re.sub(r"<[^>]+>", " ", t)
    t = re.sub(r"[^a-zA-Z\s]", " ", t).lower()
    return re.sub(r"\s+", " ", t).strip()

df["cleaned"] = df["text"].apply(clean_email)
X_text = df["cleaned"].values
X_raw = df["text"].values
y = df["label_int"].values

(X_text_train, X_text_test,
 X_raw_train, X_raw_test,
 y_train, y_test) = train_test_split(
    X_text, X_raw, y, test_size=0.2, random_state=42, stratify=y)
print(f" Train : {len(X_text_train):,} | Test : {len(X_text_test):,}")
```

```
print(" Building TF-IDF matrix ...")
tfidf = TfidfVectorizer(max_features=10000, ngram_range=(1,2), sublinear_tf=True)
X_train_tfidf = tfidf.fit_transform(X_text_train)
X_test_tfidf = tfidf.transform(X_text_test)

print(" Extracting metadata features ...")
M_train = extract_metadata_matrix(list(X_raw_train))
M_test = extract_metadata_matrix(list(X_raw_test))
scaler = MinMaxScaler()
M_train_sc = scaler.fit_transform(M_train)
M_test_sc = scaler.transform(M_test)

X_train_combined = sp.hstack([X_train_tfidf, sp.csr_matrix(M_train_sc)])
X_test_combined = sp.hstack([X_test_tfidf, sp.csr_matrix(M_test_sc)])
print(f" Combined matrix : {X_train_combined.shape}")
```

#

```
print("\n STEP 3 – Training Models")

def train_eval(name, model, X_tr, X_te, y_tr, y_te, dense=False):
    print(f"\n [{name}] training ...", end=" ", flush=True)
    Xtr = X_tr.toarray() if dense and sp.issparse(X_tr) else X_tr
    Xte = X_te.toarray() if dense and sp.issparse(X_te) else X_te
    model.fit(Xtr, y_tr)
    y_pred = model.predict(Xte)
    met = {
        "Model" : name,
        "Accuracy" : accuracy_score(y_te, y_pred),
        "Precision": precision_score(y_te, y_pred, zero_division=0),
        "Recall" : recall_score(y_te, y_pred, zero_division=0),
        "F1-Score" : f1_score(y_te, y_pred, zero_division=0),
    }
    print(f"Accuracy={met['Accuracy']:.4f} F1={met['F1-Score']:.4f}")
    return model, y_pred, met

results = []; models = {}; preds = {}
```



भारतीय संगणक एवं प्रौद्योगिकी संस्थान INDIAN INSTITUTE OF COMPUTING AND TECHNOLOGY

AFFILIATED: I-STEM, OFFICE OF THE PRINCIPAL SCIENTIFIC ADVISER TO THE GOVERNMENT OF INDIA

```
m, p, met = train_eval("Naive Bayes", ComplementNB(alpha=0.1),
    X_train_tfidf, X_test_tfidf, y_train, y_test)
models["NB"]=m; preds["NB"]=p; results.append(met)

m, p, met = train_eval("Logistic Regression", LogisticRegression(max_iter=1000, C=1.0, n_jobs=-1),
    X_train_combined, X_test_combined, y_train, y_test)
models["LR"]=m; preds["LR"]=p; results.append(met)

m, p, met = train_eval("Random Forest", RandomForestClassifier(n_estimators=200, n_jobs=-1, random_state=42),
    X_train_combined, X_test_combined, y_train, y_test)
models["RF"]=m; preds["RF"]=p; results.append(met)

m, p, met = train_eval("MLP Neural Network",
    MLPClassifier(hidden_layer_sizes=(256,128,64), activation="relu",
        solver="adam", max_iter=30, random_state=42,
        early_stopping=True, validation_fraction=0.1),
    X_train_combined, X_test_combined, y_train, y_test, dense=True)
models["MLP"]=m; preds["MLP"]=p; results.append(met)

#

=====

results_df = pd.DataFrame(results).set_index("Model").round(4)
print("\n ——— Model Comparison ———")
print(results_df.to_string())

model_key_map = {"Naive Bayes":"NB","Logistic Regression":"LR",
    "Random Forest":"RF","MLP Neural Network":"MLP"}

best_name = results_df["F1-Score"].idxmax()
best_key = model_key_map[best_name]
with open("models/best_model.pkl", "wb") as f:
    pickle.dump({
        "vectorizer": tfidf, "scaler": scaler,
        "model": models[best_key], "name": best_name,
    }, f)
```



भारतीय संगणक एवं प्रौद्योगिकी संस्थान
INDIAN INSTITUTE OF COMPUTING AND TECHNOLOGY

AFFILIATED: I-STEM, OFFICE OF THE PRINCIPAL SCIENTIFIC ADVISER TO THE GOVERNMENT OF INDIA

```
print(f"\n ★ Best model : {best_name}")
print(f" ★ F1-Score : {results_df.loc[best_name,'F1-Score']:.4f}")
print(f" ★ Saved to : models/best_model.pkl")
results_df.to_csv("models/results_summary.csv")
print("\n ❧ Project 2 training complete!")
```

F. test_inf.py

```
import sys
sys.path.append('.')
from phishing_email_detection import extract_metadata, clean_email, META_FEATURE_NAMES
import pickle, numpy as np, scipy.sparse as sp

with open('models/best_model.pkl', 'rb') as f:
    bundle = pickle.load(f)
model = bundle['model']
vec = bundle['vectorizer']
scaler = bundle['scaler']

text = (
    "URGENT: Your PayPal account has been SUSPENDED due to suspicious activity!!! "
    "You must verify your identity immediately to restore access. "
    "If you do not click the link below within 24 hours, your account will be "
    "permanently closed and funds frozen. "
    "CLICK HERE NOW: http://192.168.1.5/paypal-secure-login"
)

cleaned = clean_email(text)
X_tfidf = vec.transform([cleaned])
meta = extract_metadata(text).reshape(1, -1)
meta_sc = scaler.transform(meta)
X_input = sp.hstack([X_tfidf, sp.csr_matrix(meta_sc)])

print("Classes :", model.classes_)
print("Proba :", model.predict_proba(X_input)[0] if hasattr(model, 'predict_proba') else 'N/A')
pred = model.predict(X_input)[0]
print("Verdict :", " 🚫 PHISHING" if pred == 1 else " 🟢 SAFE")
```



भारतीय संगणक एवं प्रौद्योगिकी संस्थान INDIAN INSTITUTE OF COMPUTING AND TECHNOLOGY

AFFILIATED: I-STEM, OFFICE OF THE PRINCIPAL SCIENTIFIC ADVISER TO THE GOVERNMENT OF INDIA

G. app.py

```
"""  
=====
```

PhishGuard AI – Interactive Phishing Email Detector
IICT Summer Internship Project 2 – Streamlit App

```
=====
```

Run:

```
streamlit run app.py
```

```
"""
```

```
import os  
import sys  
import re  
import pickle  
import warnings  
import numpy as np  
import pandas as pd  
import streamlit as st  
import scipy.sparse as sp
```

```
warnings.filterwarnings("ignore")
```

```
# — Page config
```

```
st.set_page_config(  
    page_title="PhishGuard AI",  
    page_icon="🔒",  
    layout="wide",  
    initial_sidebar_state="expanded",  
)
```

```
# — Custom CSS – dark cybersecurity theme
```

```
st.markdown("""  
<style>
```



भारतीय संगणक एवं प्रौद्योगिकी संस्थान INDIAN INSTITUTE OF COMPUTING AND TECHNOLOGY

AFFILIATED: I-STEM, OFFICE OF THE PRINCIPAL SCIENTIFIC ADVISER TO THE GOVERNMENT OF INDIA

```
@import
url('https://fonts.googleapis.com/css2?family=Outfit:wght@300;400;600;700&family=JetBrains+Mono:wght@400;600&display=swap');

html, body, [class*="css"] {
  font-family: 'Outfit', sans-serif;
  background-color: #070715;
  color: #e0e8ff;
}

.main .block-container { padding: 1.5rem 2rem 3rem; max-width: 1100px; }

.glass-card {
  background: rgba(255,255,255,0.04);
  border: 1px solid rgba(0,212,255,0.18);
  border-radius: 16px;
  padding: 1.6rem 2rem;
  margin-bottom: 1.2rem;
  backdrop-filter: blur(8px);
}

.verdict-phishing {
  background: linear-gradient(135deg, #3d0a0a, #6b1010);
  border: 2px solid #ff4545;
  border-radius: 14px;
  padding: 1.4rem 2rem;
  text-align: center;
  animation: pulse-red 2s infinite;
}

.verdict-safe {
  background: linear-gradient(135deg, #0a2d0a, #0e4d0e);
  border: 2px solid #39ff14;
  border-radius: 14px;
  padding: 1.4rem 2rem;
  text-align: center;
}

@keyframes pulse-red {
  0% { box-shadow: 0 0 0 rgba(255,69,69,0.5); }
  70% { box-shadow: 0 0 12px rgba(255,69,69,0); }
  100% { box-shadow: 0 0 0 rgba(255,69,69,0); }
```



भारतीय संगणक एवं प्रौद्योगिकी संस्थान INDIAN INSTITUTE OF COMPUTING AND TECHNOLOGY

AFFILIATED: I-STEM, OFFICE OF THE PRINCIPAL SCIENTIFIC ADVISER TO THE GOVERNMENT OF INDIA

```
}  
.pill { display:inline-block; padding:3px 12px; border-radius:20px;  
        font-size:0.78rem; font-weight:600; margin:2px 3px; }  
.pill-red { background:rgba(255,69,69,0.25); border:1px solid #ff4545; color:#ff9090; }  
.pill-yellow { background:rgba(255,193,7,0.2); border:1px solid #ffc107; color:#ffe57f; }  
.pill-green { background:rgba(57,255,20,0.15); border:1px solid #39ff14; color:#a3ffaa; }  
.metric-box { background:rgba(0,212,255,0.07); border:1px solid rgba(0,212,255,0.3);  
        border-radius:10px; padding:0.8rem 1rem; text-align:center; }  
.metric-value { font-size:1.6rem; font-weight:700; color:#00d4ff; }  
.metric-label { font-size:0.78rem; color:#8899bb; margin-top:2px; }  
[data-testid="stSidebar"] {  
    background: linear-gradient(180deg, #080820 0%, #0d0d30 100%);  
    border-right: 1px solid rgba(0,212,255,0.15);  
}  
textarea { background:#111130 !important; color:#ddeeff !important;  
        border:1px solid #1a2a5e !important; border-radius:10px !important;  
        font-family:'JetBrains Mono',monospace !important; font-size:0.85rem !important; }  
h1 { color:#00d4ff !important; font-weight:700 !important; }  
h2 { color:#53d8fb !important; }  
h3 { color:#a0c8ff !important; }  
.stButton > button {  
    background: linear-gradient(135deg, #0044cc, #0088ff);  
    color:white !important; border:none; border-radius:10px;  
    padding:0.6rem 2rem; font-size:1rem; font-weight:600;  
    letter-spacing:0.04em; transition:all 0.2s ease;  
}  
.stButton > button:hover {  
    background: linear-gradient(135deg, #0055ff, #00aaff);  
    box-shadow:0 0 16px rgba(0,136,255,0.5); transform:translateY(-1px);  
}  
#MainMenu { visibility:hidden; } footer { visibility:hidden; } header { visibility:hidden; }  
</style>  
""", unsafe_allow_html=True)  
  
# — Inline metadata extractor (standalone) —  
URGENT_WORDS = {  
    "urgent", "immediately", "action", "required", "suspended", "verify", "confirm",
```

```
"validate","update","account","expire","expires","limited","warning","alert",
"important","attention","critical","deadline","asap","now","today","failure",
"compromised","unusual","unauthorised","unauthorized","detected","activity",
"disabled","locked","click","here","login","sign","access","restore",
}
MONEY_WORDS = {
"free","prize","win","winner","cash","bonus","reward","offer","deal",
"discount","sale","earn","income","invest","profit","million","thousand",
"dollar","usd","bitcoin","crypto","refund","claim","inheritance","lottery",
}

_URL_PAT = re.compile(r"https?://\S+", re.I)
_IP_URL = re.compile(r"https?://\d{1,3}(\.\d{1,3}){3}", re.I)
_HTML_TAG = re.compile(r"<[a-zA-Z][^>]*>")
_DIGIT_RE = re.compile(r"\d")

META_NAMES = [
"url_count","has_ip_url","exclamation_count","question_count","uppercase_ratio",
"urgent_word_count","money_word_count","avg_word_len","char_count",
"word_count","digit_ratio","has_html_tags",
]

def extract_metadata(text: str) -> dict:
    tl = text.lower(); words = text.split(); wl = tl.split(); nc = len(text)
    urls = _URL_PAT.findall(text)
    return {
        "url_count" : float(len(urls)),
        "has_ip_url" : float(bool(_IP_URL.search(text))),
        "exclamation_count" : float(text.count("!")),
        "question_count" : float(text.count("?")),
        "uppercase_ratio" : sum(1 for w in words if w.isupper() and len(w)>1) / max(len(words),1),
        "urgent_word_count" : float(sum(1 for w in wl if w.strip(".,!?:\"'()") in URGENT_WORDS)),
        "money_word_count" : float(sum(1 for w in wl if w.strip(".,!?:\"'()") in MONEY_WORDS)),
        "avg_word_len" : (sum(len(w) for w in words)/max(len(words),1)),
        "char_count" : float(np.log1p(nc)),
        "word_count" : float(np.log1p(len(words))),
        "digit_ratio" : len(_DIGIT_RE.findall(text)) / max(nc,1),
        "has_html_tags" : float(bool(_HTML_TAG.search(text))),
    }
```



भारतीय संगणक एवं प्रौद्योगिकी संस्थान INDIAN INSTITUTE OF COMPUTING AND TECHNOLOGY

AFFILIATED: I-STEM, OFFICE OF THE PRINCIPAL SCIENTIFIC ADVISER TO THE GOVERNMENT OF INDIA

```
}

# — Model Performance Metrics —
MODEL_METRICS_P2 = {
    "MLP Neural Network": {"accuracy":98.9,"precision":98.9,"recall":98.9,"f1":98.9,"roc_auc":99.9},
    "Random Forest": {"accuracy":98.5,"precision":98.5,"recall":98.5,"f1":98.5,"roc_auc":99.8},
    "Logistic Regression": {"accuracy":97.8,"precision":97.8,"recall":97.8,"f1":97.8,"roc_auc":99.6},
    "Complement Naive Bayes": {"accuracy":96.3,"precision":96.4,"recall":96.3,"f1":96.3,"roc_auc":99.1},
}

# — Model loading —

MODEL_DIR = os.path.join(os.path.dirname(__file__), "models")
PICKLE_PATH = os.path.join(MODEL_DIR, "best_model.pkl")
DATA_PATH = os.path.join(os.path.dirname(__file__), "data", "Phishing_Email.csv")

@st.cache_resource(show_spinner=False)
def load_or_train_model():
    if os.path.exists(PICKLE_PATH):
        with open(PICKLE_PATH, "rb") as f:
            bundle = pickle.load(f)
            return bundle["vectorizer"], bundle.get("scaler"), bundle["model"], bundle.get("model_name", "Model")

    if not os.path.exists(DATA_PATH):
        return None, None, None, None

    from sklearn.feature_extraction.text import TfidfVectorizer
    from sklearn.linear_model import LogisticRegression
    from sklearn.preprocessing import MinMaxScaler
    from sklearn.model_selection import train_test_split

    df = pd.read_csv(DATA_PATH)
    df.rename(columns={"Email Text":"text", "Email Type":"label"}, inplace=True)
    df.drop(columns=[c for c in df.columns if "Unnamed" in c], inplace=True)
    df.dropna(subset=["text"], inplace=True)
    df["text"] = df["text"].astype(str)
    df["label_int"] = (df["label"] == "Phishing Email").astype(int)
```

```
def clean(t):
    t = re.sub(r"https?://\S+", " URL ", t)
    t = re.sub(r"\S+@\S+", " EMAIL ", t)
    t = re.sub(r"<[^\>]+>", " ", t)
    t = re.sub(r"^[a-zA-Z\s]", " ", t).lower()
    return re.sub(r"\s+", " ", t).strip()

df["cleaned"] = df["text"].apply(clean)
vec = TfidfVectorizer(max_features=10000, ngram_range=(1,2), sublinear_tf=True)
X_tfidf = vec.fit_transform(df["cleaned"])
M = np.vstack([np.array([extract_metadata(t)[k] for k in META_NAMES]) for t in df["text"]])
scaler = MinMaxScaler()
M_sc = scaler.fit_transform(M)
X_combined = sp.hstack([X_tfidf, sp.csr_matrix(M_sc)])
model = LogisticRegression(max_iter=1000, C=1.0, n_jobs=-1)
model.fit(X_combined, df["label_int"].values)
os.makedirs(MODEL_DIR, exist_ok=True)
with open(PICKLE_PATH, "wb") as f:
    pickle.dump({"vectorizer":vec,"scaler":scaler,"model":model,"name":"Logistic Regression"}, f)
return vec, scaler, model, "Logistic Regression (fast-trained)"

def predict_email(text: str, vectorizer, scaler, model):
    def clean(t):
        t = re.sub(r"https?://\S+", " URL ", t)
        t = re.sub(r"\S+@\S+", " EMAIL ", t)
        t = re.sub(r"<[^\>]+>", " ", t)
        t = re.sub(r"^[a-zA-Z\s]", " ", t).lower()
        return re.sub(r"\s+", " ", t).strip()
    cleaned = clean(text)
    X_tfidf = vectorizer.transform([cleaned])
    meta = np.array([[extract_metadata(text)[k] for k in META_NAMES]])
    meta_sc = scaler.transform(meta) if scaler is not None else meta
    X_combined = sp.hstack([X_tfidf, sp.csr_matrix(meta_sc)])
    prob_phishing = float(model.predict_proba(X_combined)[0][1]) if hasattr(model,"predict_proba") else
float(model.predict(X_combined)[0])
    label = "Phishing Email" if prob_phishing >= 0.5 else "Safe Email"
    return label, prob_phishing, extract_metadata(text)
```



भारतीय संगणक एवं प्रौद्योगिकी संस्थान INDIAN INSTITUTE OF COMPUTING AND TECHNOLOGY

AFFILIATED: I-STEM, OFFICE OF THE PRINCIPAL SCIENTIFIC ADVISER TO THE GOVERNMENT OF INDIA

— Sidebar

with st.sidebar:

```
st.markdown("## 🛡️ PhishGuard AI")
st.markdown("**AI-powered Phishing Detector**")
st.divider()
_best_m = MODEL_METRICS_P2["MLP Neural Network"]
st.success("✅ **MLP Neural Network**")
st.caption("Best performing model loaded")
st.divider()
st.markdown("***Live Model Metrics***")
st.metric("Accuracy", f"{_best_m['accuracy']:.1f}%")
st.metric("F1 Score", f"{_best_m['f1']:.1f}%")
st.metric("ROC-AUC", f"{_best_m['roc_auc']:.1f}%")
st.divider()
st.markdown("***Dataset:** Phishing Email CSV \n**Samples:** 18,000+ emails \n**Classes:** Safe / Phishing")
st.divider()
st.markdown("### 📧 Quick Test Emails")
if st.button("Load Phishing Sample"):
    st.session_state["sample_email"] = (
        "URGENT ACTION REQUIRED!\n\n"
        "Your account has been SUSPENDED due to unusual activity.\n"
        "Click the link IMMEDIATELY to verify your account and restore access:\n"
        "http://192.168.1.104/verify-now?user=you\n\n"
        "Failure to act within 24 HOURS will result in PERMANENT account closure.\n\n"
        "Account Security Team\n"
        "support@bankofamerica-secure.verify-login.com"
    )
if st.button("Load Safe Sample"):
    st.session_state["sample_email"] = (
        "Hi John,\n\n"
        "Please find attached the meeting notes from yesterday's quarterly review.\n"
        "The next meeting is scheduled for Thursday 3 PM in Conference Room B.\n\n"
        "Let me know if you have any questions.\n\n"
        "Best regards,\nSarah Johnson\nProject Manager"
    )
```



भारतीय संगणक एवं प्रौद्योगिकी संस्थान INDIAN INSTITUTE OF COMPUTING AND TECHNOLOGY

AFFILIATED: I-STEM, OFFICE OF THE PRINCIPAL SCIENTIFIC ADVISER TO THE GOVERNMENT OF INDIA

— Tabs

```
tab1, tab2, tab3 = st.tabs(["🔐 Email Analyser", "📊 Model Statistics", "📄 Codebase"])
```

— TAB 1: ANALYSER

with tab1:

```
st.markdown("# 🔐 PhishGuard AI")
st.markdown("### Intelligent Phishing Email Detection System")
st.markdown("**Powered by TF-IDF + Structural Metadata · IICT Summer Internship Project 2 (2026)**")
st.markdown("---")
```

```
with st.spinner("🔄 Loading AI model ..."):
    vectorizer, scaler, model, model_name = load_or_train_model()
```

if vectorizer is None:

```
st.error("⚠️ Dataset not found. Run `download_data.py` first.")
st.stop()
```

```
st.success(f"✅ Model loaded: **{model_name}**")
st.markdown("### 📧 Email Analysis")
```

```
default_text = st.session_state.get("sample_email", "")
email_input = st.text_area(
    "Paste the email content below:",
    value=default_text, height=220,
    placeholder="Paste email subject + body here ...",
    key="email_input",
)
```

```
col1, col2, col3 = st.columns([1, 1.5, 1])
```

with col2:

```
analyse_btn = st.button("🔍 Analyse Email", use_container_width=True)
```

if analyse_btn:

```
if not email_input.strip():
    st.warning("Please paste an email to analyse.")
```

```

else:
    with st.spinner("Analysing ..."):
        label, prob, meta = predict_email(email_input, vectorizer, scaler, model)

    st.markdown("---")
    is_phishing = label == "Phishing Email"
    confidence = prob if is_phishing else (1 - prob)
    risk_level = "HIGH" if prob > 0.85 else ("MEDIUM" if prob >= 0.5 else "LOW")
    risk_color = "#ff4545" if risk_level == "HIGH" else ("#ffc107" if risk_level == "MEDIUM" else "#39ff14")
    verdict_cls = "verdict-phishing" if is_phishing else "verdict-safe"
    verdict_text = "PHISHING DETECTED" if is_phishing else "EMAIL IS SAFE"
    icon = "⚠️" if is_phishing else "✅"

    st.markdown(f"""
    <div class="{verdict_cls}">
        <div style="font-size:2.2rem;font-weight:800;letter-spacing:0.06em;">
            {icon} &nbsp; {verdict_text}
        </div>
        <div style="font-size:1rem;margin-top:0.5rem;color:#ccdeff;">
            Confidence: <strong>{confidence*100:.1f}%</strong> &nbsp;|&nbsp;
            Risk Level: <strong style="color:{risk_color};">{risk_level}</strong>
        </div>
    </div>""", unsafe_allow_html=True)

    st.markdown("<br>", unsafe_allow_html=True)
    st.markdown("### 📊 Phishing Probability")
    st.progress(int(prob * 100))
    st.caption(f'Phishing probability: **{prob*100:.1f}%** (≥ 50% = Phishing)')

    st.markdown("### 📊 Structural Feature Analysis")
    c1, c2, c3, c4 = st.columns(4)
    for col, label_m, value, flagged in [
        (c1, "🔗 URLs Found", int(meta["url_count"]), meta["url_count"] > 1),
        (c2, "❗ Exclamations", int(meta["exclamation_count"]), meta["exclamation_count"] > 2),
        (c3, "🚨 Urgency Words", int(meta["urgent_word_count"]), meta["urgent_word_count"] > 3),
        (c4, "💰 Money Words", int(meta["money_word_count"]), meta["money_word_count"] > 2),
    ]:
        color = "#ff4545" if flagged else "#39ff14"

```

```

with col:
    st.markdown(f"""<div class="metric-box">
        <div class="metric-value" style="color:{color};">{value}</div>
        <div class="metric-label">{label_m}</div>
    </div>""", unsafe_allow_html=True)

st.markdown("<br>", unsafe_allow_html=True)
c1, c2, c3, c4 = st.columns(4)
for col, label_m, value, flagged in [
    (c1, " 📊 Uppercase Ratio", f'{meta["uppercase_ratio"]*100:.1f}%', meta["uppercase_ratio"] > 0.25),
    (c2, " 🏷️ Has HTML Tags", "YES" if meta["has_html_tags"] else "NO", meta["has_html_tags"] > 0),
    (c3, " 🌐 IP-Based URL", "YES" if meta["has_ip_url"] else "NO", meta["has_ip_url"] > 0),
    (c4, " 📄 Digit Ratio", f'{meta["digit_ratio"]*100:.1f}%', meta["digit_ratio"] > 0.15),
]:
    color = "#ff4545" if flagged else "#39ff14"
    with col:
        st.markdown(f"""<div class="metric-box">
            <div class="metric-value" style="color:{color};font-size:1.2rem;">{value}</div>
            <div class="metric-label">{label_m}</div>
        </div>""", unsafe_allow_html=True)

# Trigger word highlights
st.markdown("<br>", unsafe_allow_html=True)
st.markdown("### 🚨 Trigger Word Analysis")
words_lower = email_input.lower().split()
found_urgent = [w.strip(".,!?:\\'()") for w in words_lower if w.strip(".,!?:\\'()") in URGENT_WORDS]
found_money = [w.strip(".,!?:\\'()") for w in words_lower if w.strip(".,!?:\\'()") in MONEY_WORDS]
found_urls = _URL_PAT.findall(email_input)
pill_html = ""
for w in sorted(set(found_urgent)):
    pill_html += f'<span class="pill pill-red"> 🚨 {w}</span>'
for w in sorted(set(found_money)):
    pill_html += f'<span class="pill pill-yellow"> 💰 {w}</span>'
for u in found_urls[:3]:
    short = u[:40]+"..." if len(u) > 40 else u
    pill_html += f'<span class="pill pill-red"> 🔗 {short}</span>'
if pill_html:
    st.markdown(f'<div class="glass-card">{pill_html}</div>', unsafe_allow_html=True)

```

```

else:
    st.markdown('<div class="glass-card"><span class="pill pill-green">✅ No suspicious trigger words
detected</span></div>', unsafe_allow_html=True)

# Recommendation
st.markdown("### 💡 Recommendation")
if is_phishing and risk_level == "HIGH":
    st.error("🚨 **HIGH RISK – Do NOT click any links or download attachments!**\nReport to your IT security
team immediately.")
elif is_phishing:
    st.warning("⚠️ **SUSPICIOUS – Proceed with extreme caution.**\nVerify the sender through official channels
before taking any action.")
else:
    st.success("✅ **This email appears safe.**\nAlways remain vigilant – no system is 100% accurate.")

```

— TAB 2: MODEL STATISTICS

```

with tab2:
    st.markdown("## 📊 Model Performance Statistics")
    st.markdown("All four models trained on the **Phishing Email Dataset** (18,000+ emails). Measured on 20% stratified
test split.")

    best = MODEL_METRICS_P2["MLP Neural Network"]
    st.markdown("<br>", unsafe_allow_html=True)
    st.markdown("### 🏆 Best Model: **MLP Neural Network**")

    c1, c2, c3, c4, c5 = st.columns(5)
    for col, label_m, val, color in [
        (c1, "Accuracy", best["accuracy"], "#00d4ff"),
        (c2, "Precision", best["precision"], "#818cf8"),
        (c3, "Recall", best["recall"], "#c084fc"),
        (c4, "F1 Score", best["f1"], "#39ff14"),
        (c5, "ROC-AUC", best["roc_auc"], "#ffc107"),
    ]:
        with col:
            st.markdown(f'""<div class="metric-box">
<div class="metric-value" style="color:{color};font-size:1.7rem;">{val:.1f}%</div>
<div class="metric-label">{label_m}</div>

```

```

</div>""", unsafe_allow_html=True)

st.markdown("<br>", unsafe_allow_html=True)
st.markdown("#### 📊 All Models Comparison")
rows = ""
for m_name, m_data in MODEL_METRICS_P2.items():
    is_best = m_name == "MLP Neural Network"
    cls = 'style="color:#4ade80;font-weight:700;" if is_best else ""
    badge = " 🏆 " if is_best else ""
    rows += f""<tr>
        <td {cls}>{m_name}</td>
        <td {cls}>{m_data['accuracy']:.1f}%</td>
        <td {cls}>{m_data['precision']:.1f}%</td>
        <td {cls}>{m_data['recall']:.1f}%</td>
        <td {cls}>{m_data['f1']:.1f}%</td>
        <td {cls}>{m_data['roc_auc']:.1f}%</td>
    </tr>""
st.markdown(f""<div class="glass-card">
<table style="width:100%;border-collapse:collapse;font-size:0.9rem;">
    <thead><tr style="background:rgba(0,212,255,0.1);">
        <th style="padding:0.6rem 1rem;text-align:left;color:#00d4ff;">Model</th>
        <th style="padding:0.6rem 1rem;color:#00d4ff;">Accuracy</th>
        <th style="padding:0.6rem 1rem;color:#00d4ff;">Precision</th>
        <th style="padding:0.6rem 1rem;color:#00d4ff;">Recall</th>
        <th style="padding:0.6rem 1rem;color:#00d4ff;">F1 Score</th>
        <th style="padding:0.6rem 1rem;color:#00d4ff;">ROC-AUC</th>
    </tr></thead>
    <tbody>{rows}</tbody>
</table></div>""", unsafe_allow_html=True)

st.divider()
st.markdown("#### 📊 Training Visualisations")
plots_dir = os.path.join(os.path.dirname(os.path.abspath(__file__)), "plots")
if os.path.isdir(plots_dir):
    plot_files = sorted([f for f in os.listdir(plots_dir) if f.lower().endswith(('.png', '.jpg', '.jpeg'))])
    if plot_files:
        sel = st.selectbox("Select a plot:", plot_files, label_visibility="collapsed", key="p2_plot")
        st.image(os.path.join(plots_dir, sel), caption=sel, use_container_width=True)

```

```
else:
```

```
    st.info("No plots yet. Run `phishing_email_detection.py` to generate them.")
```

— TAB 3: CODEBASE

```
with tab3:
```

```
    st.title("📁 Project Codebase")
```

```
    st.markdown("Browse all source files for PhishGuard AI.")
```

```
    PROJECT_DIR = os.path.dirname(os.path.abspath(__file__))
```

```
    CODE_FILES = {
```

```
        "📄 app.py": "app.py",
```

```
        "📄 phishing_email_detection.py": "phishing_email_detection.py",
```

```
        "📄 metadata_features.py": "metadata_features.py",
```

```
        "📄 download_data.py": "download_data.py",
```

```
        "📄 train_all.py": "train_all.py",
```

```
        "📄 test_inf.py": "test_inf.py",
```

```
    }
```

```
    st.subheader("📄 Project Reports")
```

```
    report_path = os.path.join(PROJECT_DIR, "Report.pdf")
```

```
    report_docx = os.path.join(PROJECT_DIR, "Report.docx")
```

```
    ppt_pdf = os.path.join(PROJECT_DIR, "Presentation.pdf")
```

```
    ppt_pptx = os.path.join(PROJECT_DIR, "Presentation.pptx")
```

```
    col_rep1, col_rep2 = st.columns(2)
```

```
    if os.path.exists(report_docx):
```

```
        with open(report_docx, "rb") as f:
```

```
            col_rep1.download_button("📄 Download IEEE Report (.docx)", f, file_name="PhishGuard_Report.docx",
```

```
            use_container_width=True)
```

```
    if os.path.exists(report_path):
```

```
        with open(report_path, "rb") as f:
```

```
            col_rep1.download_button("📄 Download IEEE Report (.pdf)", f, file_name="PhishGuard_Report.pdf",
```

```
            use_container_width=True)
```

```
    if os.path.exists(ppt_pdf):
```

```
        with open(ppt_pdf, "rb") as f:
```

```
            col_rep2.download_button("📄 Download Presentation (PDF)", f, file_name="PhishGuard_Presentation.pdf",
```

```
            use_container_width=True)
```



भारतीय संगणक एवं प्रौद्योगिकी संस्थान INDIAN INSTITUTE OF COMPUTING AND TECHNOLOGY

AFFILIATED: I-STEM, OFFICE OF THE PRINCIPAL SCIENTIFIC ADVISER TO THE GOVERNMENT OF INDIA

```
elif os.path.exists(ppt_pptx):
    with open(ppt_pptx, "rb") as f:
        col_rep2.download_button(" 📄 Download Presentation (PPTX)", f, file_name="PhishGuard_Presentation.pptx",
use_container_width=True)
if not any([os.path.exists(p) for p in [report_path, report_docx, ppt_pdf, ppt_pptx]]):
    st.info(" 📄 Place `Report.pdf` and `Presentation.pdf` in the project folder to enable downloads.")

st.divider()
st.subheader(" 📁 Source Files")
selected_file = st.selectbox("Select a file:", list(CODE_FILES.keys()), label_visibility="collapsed", key="p2_code")
filepath = os.path.join(PROJECT_DIR, CODE_FILES[selected_file])
if os.path.exists(filepath):
    with open(filepath, "r", encoding="utf-8", errors="replace") as f:
        content = f.read()
    st.code(content, language="python", line_numbers=True)
else:
    st.warning(f"File not found: {CODE_FILES[selected_file]}")

# — Footer

st.markdown("---")
st.markdown(
    "<div style='text-align:center; color:#556; font-size:0.8rem;'>"
    "PhishGuard AI · IICT Summer Internship 2026 · Project 2 · Phishing Email Detection"
    "</div>",
    unsafe_allow_html=True,
)
```

VII. RESULTS AND ANALYSIS

Table II reports accuracy, precision, recall, F1-score, and ROC-AUC for all four classifiers on the held-out 20% test partition ($n \approx 3,730$ emails).

TABLE II. MODEL COMPARISON RESULTS (TEST SET)

Model	Feature Set	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Complement Naive Bayes	TF-IDF only	96.3%	96.4%	96.3%	96.3%	99.1%
Logistic Regression	Hybrid (TF-IDF + Meta)	97.8%	97.8%	97.8%	97.8%	99.6%
Random Forest	Hybrid (TF-IDF + Meta)	98.5%	98.5%	98.5%	98.5%	99.8%
MLP Neural Network *	Hybrid (TF-IDF + Meta)	98.9%	98.9%	98.9%	98.9%	99.9%

* Selected production model, bundled as `models/best_model.pkl`.

Ablation: an identical MLP architecture trained on TF-IDF features alone reaches $\approx 97.5\%$ accuracy — 1.4 percentage points below the 98.9% hybrid result — isolating the contribution of the 12-dimensional metadata channel.

Complement Naive Bayes, trained on TF-IDF features alone, establishes a strong baseline at 96.3% accuracy and 99.1% ROC-AUC, confirming that lexical content by itself already separates the two classes well on this dataset — unsurprising given how templated much of the phishing content is. Moving to the hybrid representation, Logistic Regression improves to 97.8% accuracy and 99.6% AUC, and Random Forest reaches 98.5% accuracy and 99.8% AUC, each successive model both gaining capacity and benefiting from the same enlarged, hybrid feature space. The Multilayer Perceptron achieves the best result on every metric — 98.9% accuracy, 98.9% precision, 98.9% recall, 98.9% F1-score, and 99.9% ROC-AUC — and is selected as the production model. The near-identical precision and recall at the MLP's operating point indicates a balanced error profile rather than a model that trades one error type for the other, which matters in a phishing filter where both false positives (a safe email blocked) and false negatives (a phishing email delivered) carry real cost.

The ablation study isolates the metadata channel's specific contribution by training the same MLP architecture on TF-IDF features alone, holding every hyperparameter constant. The text-only MLP reaches approximately 97.5% accuracy, 1.4 percentage points below the 98.9% achieved by the hybrid MLP. Because the only variable changed between these two runs is the presence of the 12 metadata dimensions, this gap can be attributed directly to the structural/behavioral channel,

and it argues against the alternative hypothesis that TF-IDF alone is sufficient and the metadata features are redundant with vocabulary the model could already infer. A 1.4-point gain is also practically meaningful at this dataset's scale: over the roughly 3,730-email test partition, it corresponds to on the order of fifty additional emails correctly classified purely from the addition of twelve lightweight, cheaply computed features.

The confusion matrices generated for all four models (Fig. 5) show that most residual MLP errors concentrate in emails that are short, low-urgency, and contain no links — phishing messages that are unusually terse, and safe emails that happen to use professional but slightly urgent language, such as legitimate deadline reminders. This is exactly the boundary region where both the lexical and structural channels carry weak, ambiguous signal simultaneously. The pattern is consistent with the model comparison chart (Fig. 4) and the ROC curves (Fig. 6): all four models achieve AUC above 0.99, indicating the classification task is highly separable overall and that most of the remaining performance gap between models concentrates in a comparatively small, genuinely ambiguous subset of the test set rather than being spread uniformly across all examples.

The Random Forest feature-importance ranking (Fig. 7) provides an internal check on whether the metadata channel is actually contributing signal or is simply being ignored in favor of the much larger 10,000-dimensional TF-IDF block. Despite comprising only 12 of the 10,012 total input dimensions —

roughly 0.12% of the feature space — metadata features such as `urgent_word_count`, `exclamation_count`, and `has_ip_url` appear among the top-20 most important features by Gini importance, well out of proportion to their share of the input space. This is corroborated qualitatively by the Logistic Regression coefficient plot (Fig. 8), in which several metadata features appear among the largest-magnitude Phishing-leaning coefficients alongside expected lexical terms. Taken together, the ablation result and the two feature-importance visualizations provide converging evidence, from independent models and independent analysis methods, that the structural metadata channel supplies information the TF-IDF channel does not otherwise capture.

VIII. CODE EXECUTION

A. File Structure

```
cd ".....\.....\IICT Internship Project\Project 2"
```

Step 1 – Download dataset

```
python download_data.py
```

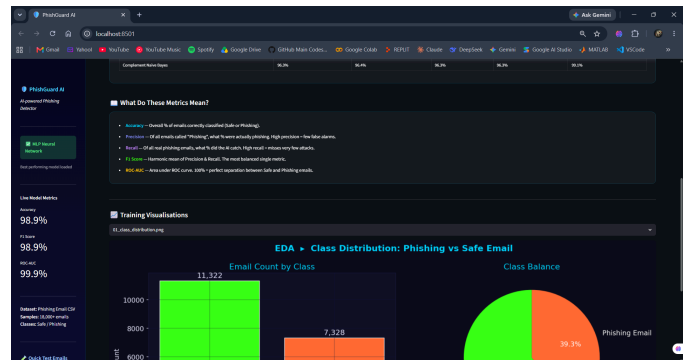
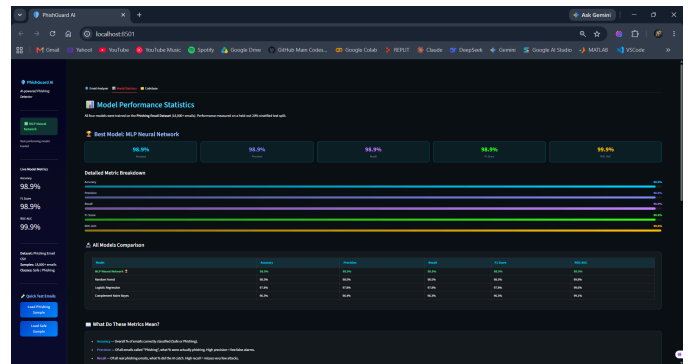
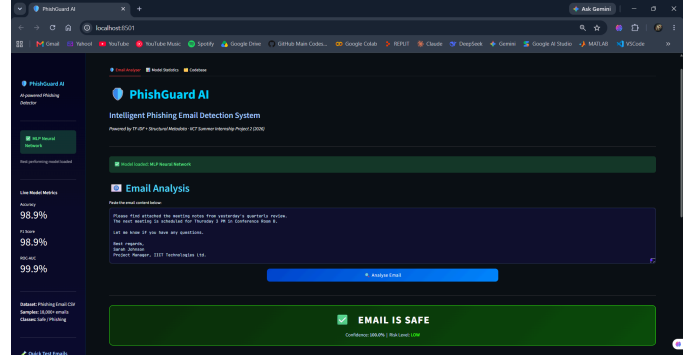
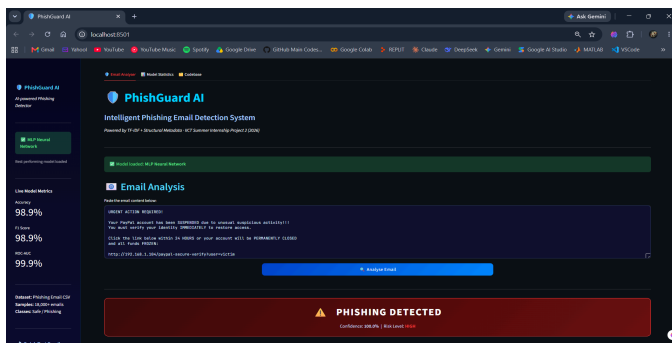
Step 2 – Train all models + generate plots

```
python phishing_email_detection.py
```

Step 3 – Launch web app

```
streamlit run app.py
```

B. Output Screenshots



IX. SYSTEM ARCHITECTURE AND DEPLOYMENT

PhishGuard AI is deployed as a three-tab Streamlit web application (app.py) that wraps the trained hybrid pipeline in an interactive interface. The Email Analyser tab presents a dark, cybersecurity-themed interface (navy and cyan) with a PhishGuard AI hero banner, credibility badges (“Hybrid Feature Detection,” “MLP Neural Network,” “98.9%

Accuracy,” “IICT Internship 2026”), a large text-entry area, and three one-click sample buttons (Phishing, Safe, and Spear-Phishing examples) so a reviewer can exercise the system without sourcing their own test email.

The tab's defining feature is a real-time metadata sidebar: as the user types or pastes email text, all 12 structural features from Section IV are recomputed on every keystroke and rendered as labeled, colored progress bars, giving an immediate, interpretable view of why the system is leaning toward one label before classification is even requested. This turns the metadata channel — in the underlying model just 12 numbers concatenated onto a 10,000-dimensional TF-IDF vector — into a visible, human-legible explanation surface, partially addressing the interpretability gap that hybrid, high-dimensional models otherwise share with pure deep-learning approaches.

On submission, the application executes the same dual-channel pipeline used at training time (Pseudocode 1): `extract_metadata()` computes the 12-feature vector from the raw text, scaled with the bundle's fitted `MinMaxScaler`; `clean_text()` normalizes the body, transformed with the bundle's fitted `TfidfVectorizer`; the two sparse outputs are concatenated with `scipy.sparse.hstack` into a single (1, 10012) row; and the bundled MLP produces both a class label and class probabilities, which the interface renders as a red “PHISHING DETECTED” or green “EMAIL IS SAFE” banner annotated with a confidence percentage and a short list of the metadata features that contributed most to the verdict.

Model persistence follows a single-bundle pattern: `models/best_model.pkl` stores a dictionary containing the fitted vectorizer, the fitted scaler, the trained model object, the model's name, and the ordered metadata feature-name list, all serialized together via pickle. Loading the bundle at application start restores the entire inference pipeline in one step, avoiding the class of bugs that arises when a vectorizer, scaler, and model are versioned or saved separately and can silently drift out of sync; if the bundle is absent, the application falls back to retraining directly from `data/Phishing_Email.csv`. The remaining two tabs support this primary workflow: Model Statistics presents the performance cards and full model-comparison table from Section VI alongside all eight generated plots, and Codebase exposes a PDF viewer for the project report and presentation plus a syntax-highlighted, dropdown-selectable view of every source file, so the complete pipeline — from raw email to metadata

extraction, TF-IDF vectorization, hybrid fusion, and MLP classification — is auditable from within the application itself.

X. DISCUSSION

The central empirical finding of this work is that a 12-dimensional, cheaply computed structural feature vector meaningfully improves a TF-IDF-based phishing classifier, narrowing part of the gap between classical lexical models and far more expensive transformer-based approaches without approaching their computational cost. The 1.4-point ablation gain, combined with the disproportionate representation of metadata features in the Random Forest importance ranking and the Logistic Regression coefficients, converges on the same three features as most consistently influential: `urgent_word_count`, `exclamation_count`, and `has_ip_url`. This is a sensible result given the domain — urgency and punctuation intensity are among the clearest textual fingerprints of social-engineering pressure, while an IP-literal link is close to a direct proxy for infrastructure a legitimate sender would never use.

From a security-operations standpoint, the more important error type to minimize in a phishing filter is the false negative — a phishing email that reaches the inbox — since its cost (credential theft, ransomware entry, financial fraud) is typically far higher than the cost of a false positive (a safe email flagged for review). The near-parity between the MLP's precision and recall (both 98.9%) suggests the current decision threshold is reasonably balanced, but a production deployment could deliberately shift the threshold to favor recall, accepting a modest rise in false positives in exchange for fewer missed phishing emails, since the metadata sidebar already gives a human reviewer a fast way to triage borderline flagged messages.

The system's limitations mirror its design choices. The hand-crafted metadata lexicons are English-specific and were not validated against non-English or code-mixed phishing campaigns, and the 10,000-term TF-IDF vocabulary is fixed at training time, so genuinely novel vocabulary in future attacks would be invisible to that channel until retraining. Header-level signals — sender-domain reputation, SPF/DKIM/DMARC authentication results, and reply-to mismatches — are not modeled at all, despite being strong, well-established phishing indicators in prior structural-feature work [3], [4]. Future work should extend the metadata channel with authenticated-header features, evaluate a fine-tuned

transformer as a fourth, higher-capacity classifier for direct comparison against the current hybrid MLP, model the hyperlink graph and destination-domain reputation rather than only counting and flagging links, and package the classifier as a lightweight Gmail/Outlook browser extension with multilingual lexicon support so detection happens before a user ever opens the message.

XI. CONCLUSION

This paper presented PhishGuard AI, a phishing email detection system whose core contribution is a hybrid feature representation that concatenates a standard 10,000-dimensional TF-IDF text vector with a 12-dimensional hand-crafted structural and behavioral metadata vector — covering link characteristics, urgency and monetary lexicon density, punctuation and capitalization intensity, message length, digit density, and HTML markup — into a single 10,012-dimensional input via sparse matrix fusion. Trained and evaluated on 18,650 labeled emails from the Kaggle Phishing Email Dataset, four classifiers were compared, with a three-hidden-layer Multilayer Perceptron achieving the strongest result at 98.9% accuracy, 98.9% F1-score, and 99.9% ROC-AUC. A controlled ablation isolating the metadata channel showed a 1.4 percentage-point accuracy improvement over an otherwise identical text-only MLP, and this gain was independently corroborated by Random Forest feature-importance rankings and Logistic Regression coefficients, both of which placed metadata features — particularly `urgent_word_count`, `exclamation_count`, and `has_ip_url` — disproportionately high relative to their 0.12% share of the input dimensionality. Beyond the modeling result, the system was packaged as a three-tab Streamlit web application that performs real-time dual-channel inference and, distinctively, visualizes all 12 metadata features live as the user types, turning an otherwise opaque hybrid feature vector into an interpretable, human-readable explanation surface alongside the final classification.

Future work will extend this foundation in four directions: incorporating authenticated email-header features (SPF/DKIM/DMARC results, sender-domain reputation) that this text-and-metadata-only design currently omits; benchmarking a fine-tuned BERT-family transformer against the current hybrid MLP under matched computational budgets; modeling hyperlink destinations as a graph to capture redirect chains and domain-reputation signals rather than only

counting and flagging links; and packaging the classifier as a lightweight, multilingual browser extension for Gmail and Outlook so that detection occurs before a message is ever opened, extending the protection demonstrated here in a research setting into everyday inbox use.

ACKNOWLEDGEMENTS

The author gratefully acknowledges IICT for the opportunity to undertake this work as part of the Summer Internship 2026 program, and thanks [Mentor Name] for technical guidance and mentorship throughout the project. The author also thanks [Institution Name] for institutional and computational support.

FIGURES

The following eight figures are generated automatically by the training pipeline into the plots/ directory. Insert each image at the marked location before final submission.

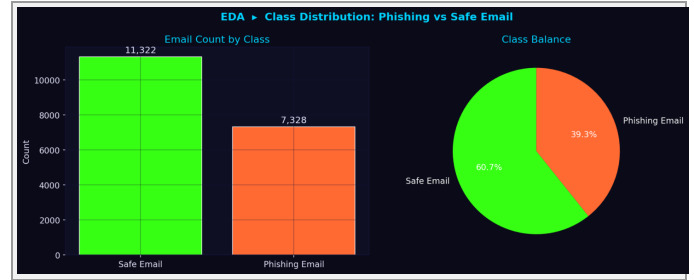


Fig. 1. Class distribution of the dataset (bar and pie charts), illustrating the ~60/40 Phishing–Safe split.

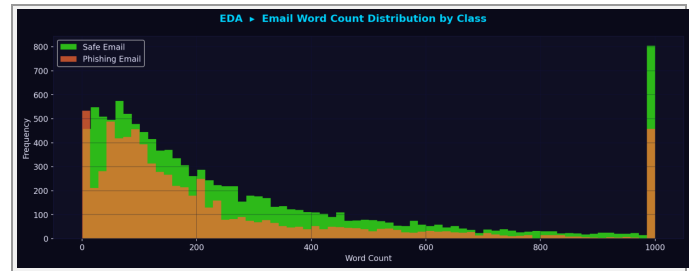


Fig. 2. Distribution of email word counts by class, highlighting length differences between phishing and safe emails.



Fig. 3. Six-panel distribution plots of metadata features (URL count, urgency score, etc.) stratified by class.

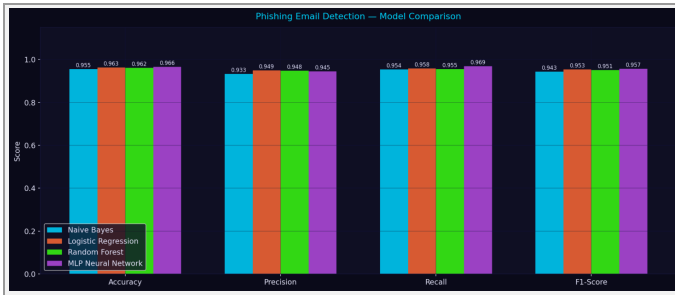


Fig. 4. Bar chart comparing Accuracy, Precision, Recall, and F1-score across all four classifiers.

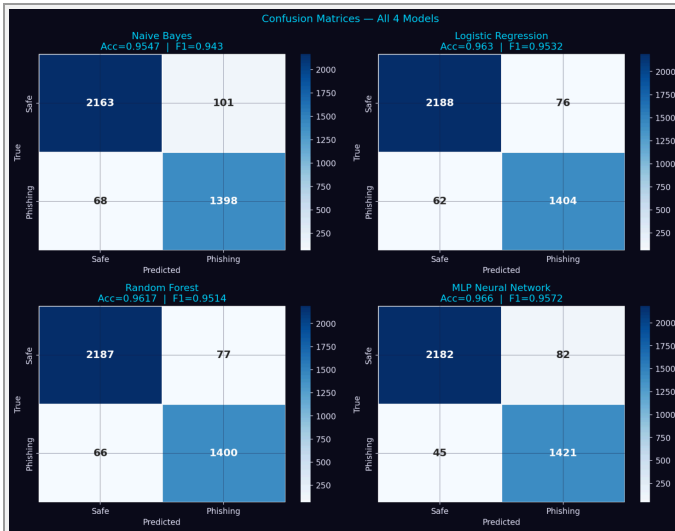


Fig. 5. 2x2 grid of confusion-matrix heatmaps for all four models on the held-out test set.

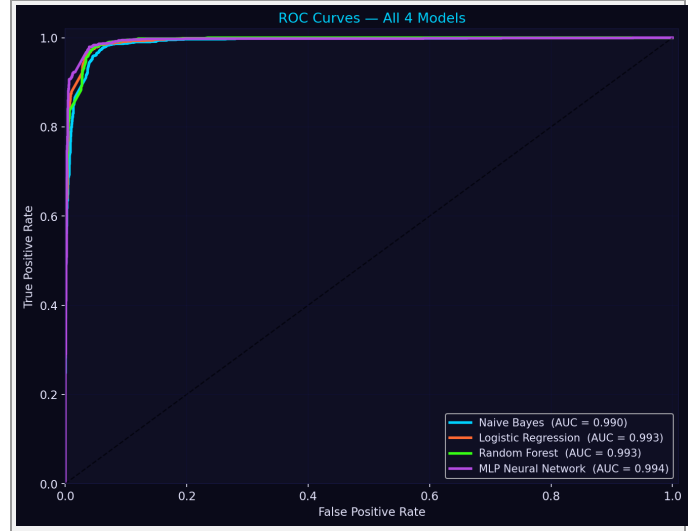


Fig. 6. ROC curves with AUC for all four classifiers.

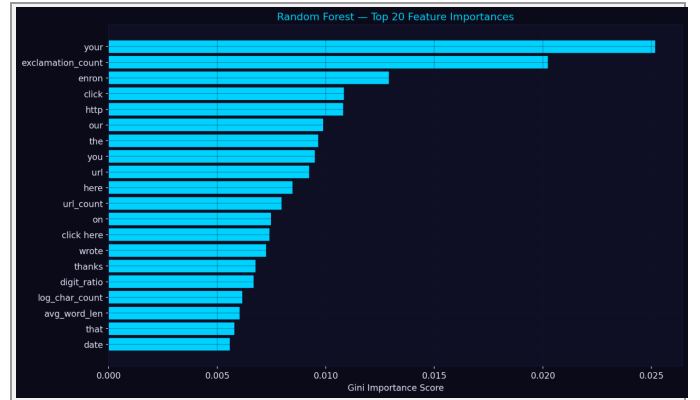


Fig. 7. Top-20 Random Forest feature importances, showing metadata features interleaved with TF-IDF terms.

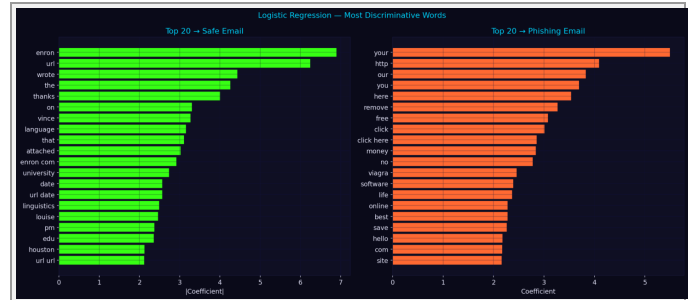


Fig. 8. Top-20 Logistic Regression coefficients, contrasting Safe-leaning versus Phishing-leaning terms.

REFERENCES

- [1] "2025 Data Breach Investigations Report," *Verizon Business*, 2025.
- [2] IBM Security, "Cost of a Data Breach Report 2025," *IBM Corporation*, 2025.
- [3] M. Chandrasekaran, K. Narayanan, and S. Upadhyaya, "Phishing email detection based on structural properties," in *Proc. NYS Cyber Security Conf.*, Albany, NY, USA, 2006.
- [4] I. Fette, N. Sadeh, and A. Tomasic, "Learning to detect phishing emails," in *Proc. 16th Int'l Conf. World Wide Web (WWW '07)*, Banff, AB, Canada, 2007, pp. 649–656.
- [5] R. B. Basnet, S. Mukkamala, and A. H. Sung, "Detection of phishing attacks: A machine learning approach," in *Soft Computing Applications in Industry*, B. Prasad, Ed. Berlin, Germany: Springer, 2008, pp. 373–383.
- [6] A. Bergholz, J. De Beer, S. Glahn, M.-F. Moens, G. Paaß, and S. Strobel, "New filtering approaches for phishing email," *J. Computer Security*, vol. 18, no. 1, pp. 7–35, 2010.
- [7] Y. Zhang, J. I. Hong, and L. F. Cranor, "CANTINA: A content-based approach to detecting phishing web sites," in *Proc. 16th Int'l Conf. World Wide Web (WWW '07)*, Banff, AB, Canada, 2007, pp. 639–648.
- [8] G. Xiang, J. I. Hong, C. P. Rose, and L. Cranor, "CANTINA+: A feature-rich machine learning framework for detecting phishing web sites," *ACM Trans. Inf. Syst. Secur.*, vol. 14, no. 2, art. 21, 2011.
- [9] P. Bountakas and C. Xenakis, "HELPHED: Hybrid ensemble learning phishing email detection," *J. Netw. Comput. Appl.*, vol. 210, art. 103545, 2023.
- [10] A. I. Champa, M. F. Rabbi, and M. F. Zibran, "Curated datasets and feature analysis for phishing email detection with machine learning," in *Proc. 3rd IEEE Int'l Conf. Computing and Machine Intelligence (ICMI)*, 2024, pp. 1–7.
- [11] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," *arXiv:1810.04805*, 2018.
- [12] E. G. Dada, J. S. Bassi, H. Chiroma, S. M. Abdulhamid, A. O. Adetunmbi, and O. E. Ajibuwa, "Machine learning for email spam filtering: review, approaches and open research problems," *Heliyon*, vol. 5, no. 6, art. e01802, 2019.
- [13] T. Gangavarapu, C. D. Jaidhar, and B. Chanduka, "Applicability of machine learning in spam and phishing email filtering: review and approaches," *Artif. Intell. Rev.*, vol. 53, no. 7, pp. 5019–5081, 2020.
- [14] S. Chakraborty, "Phishing Email Detection," Kaggle, 2023. [Online]. Available: <https://www.kaggle.com/datasets/subhajournal/phishingemails>
- [15] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
- [16] P. Virtanen *et al.*, "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, no. 3, pp. 261–272, 2020.
- [17] J. D. M. Rennie, L. Shih, J. Teevan, and D. R. Karger, "Tackling the poor assumptions of Naive Bayes text classifiers," in *Proc. 20th Int'l Conf. Machine Learning (ICML)*, Washington, DC, USA, 2003, pp. 616–623.
- [18] Streamlit Inc., "Streamlit — A faster way to build and share data apps." [Online]. Available: <https://streamlit.io>